ReviewLens

A Streamlit Text Mining and Document Intelligence System for Product-Review Analytics

Information Retrieval (Merged - AIMLCZG537 / DSECLZG537), S1-26 · Assignment 2

BITS Pilani · Work Integrated Learning Programmes Division

Group: G47

Student Registration Number	Name
2025ag05831	CHINDUKURU SAI SANDEEP
2025ag05788	B JYOTHI PRIYA
2025ag05255	PAREKH KETAN ASHWINKUMAR
2025ag05049	PULKIT KUSHWAHA
2025ag05925	SHEKHAR AGGARWAL

Submission: November 2026 (deadline 9 November 2026, 23:59)

Deliverables: Streamlit application (app.py, views/, textmining/), dataset (data/), README.md, this report, G47_Contribution.xlsx, execution evidence.

Summary

ReviewLens is an end-to-end Streamlit application that helps a marketplace's customer-experience team analyse 1,224 real product reviews from six departments (Amazon Reviews 2023). It ingests CSV, JSON or a folder of text files, runs a configurable preprocessing pipeline, builds an inverted index and length-normalised TF-IDF vectors by hand, and offers manually implemented Multinomial Naive Bayes classification, K-Means clustering with manual Purity and Rand Index, and a content-based Top-K recommender. No library implementation of TF-IDF, Naive Bayes, K-Means, Purity or Rand Index is used; the core algorithms are verified against the worked examples of Manning, Raghavan and Schütze (2008).

With the default pipeline, Naive Bayes routes **77.4%** of held-out reviews to the right department (macro-F1 0.774; 0.782 ± 0.024 over 10 random splits). K-Means with k = 6 reaches purity **0.475** and Rand Index **0.766**, against chance levels of 0.198 and 0.715. The recommender's Top-5 lists agree with the query's department 57.1% of the time (MRR 0.783). Experiments show that markup removal, stop-word removal and stemming improve classification and recommendation, that n-grams hurt on short reviews, and that duplicates inflate classification accuracy through train/test leakage (96.7% vs 77.8%).

Contents

1. Selected business context
2. Dataset description
3. Implementation approach
4. Preprocessing and feature choices
5. Classification results (Naive Bayes)
6. Clustering results (K-Means)
7. Comparison of classification and clustering
8. Recommendation results
9. Inference and discussion (compulsory questions Q1-Q5)
10. Limitations and suggested improvements
11. Evidence of execution (local machine)
12. Running the application and submission contents

References · Appendix A (additional screenshots) · Appendix B (tests)

1. Selected business context

An online marketplace receives a continuous stream of free-text product reviews across many departments. Its customer-experience (Voice-of-Customer) team has to decide, for every review, which department should look at it, has to notice recurring problems early - including problems nobody has a category for yet - and, when investigating a complaint, has to find similar earlier reviews quickly. Reading every review does not scale, and keyword search misses differently worded reviews of the same issue.

ReviewLens supports these decisions for six departments: Appliances, Beauty, Musical Instruments, Pet Supplies, Software and Video Games. Table 1 maps the analyst's questions to the techniques implemented in the system.

Table 1. *Analyst questions and the technique that answers them.*

Analyst question	Technique (manual implementation)	Output in the app
Which department should handle this review?	Multinomial Naive Bayes classification	predicted department + posterior, confusion matrix, per-class metrics
Which themes recur, without predefined labels?	K-Means clustering on TF-IDF vectors	clusters, top centroid terms, share of 1-2 star reviews per cluster
Which earlier reviews are related to this one?	Content-based Top-K recommendation (cosine)	ranked reviews with scores, metadata and shared terms
What characterises a department or its complaints?	Corpus analytics (smoothed log-odds)	distinctive terms; what negative reviews talk about
Is the same review posted twice?	Duplicate detection (hash + cosine)	duplicate groups, policy, effect on all analyses

2. Dataset description

2.1 Source and sampling

The corpus is a reproducible sample of **Amazon Reviews 2023** (McAuley Lab, UC San Diego; Hou et al., 2024), real customer reviews published for research and distributed as one JSON-lines file per product category. The script scripts/build_dataset.py (seed 537) reads 40 random 64 KB byte ranges of each of six category files, keeps English reviews of 30-350 words, allows at most two reviews per user (the files are ordered by user) and draws about 200 reviews per category. The frozen result is stored in data/ (JSON with separate metadata, plus a flat CSV) and documented in data/DATASET_CARD.md. The brief asks for at least 50-75 documents or about 5 MB of text; the corpus has 1,224 documents.

Table 2. *Corpus composition (class labels) and mean star rating.*

Category (label)	Documents	Mean rating
Appliances	201	3.84
Beauty	204	3.79
Musical Instruments	203	4.00
Pet Supplies	207	3.85
Software	206	3.82
Video Games	203	3.82
Total	1,224	

2.2 Structure: unique ID, text and separate metadata

Every document has a unique, neutral identifier (R0001 … R1224, assigned after shuffling so it never reveals the category) and one main text field. In the JSON file the metadata of each review sits in its own

"metadata" object; the ingestion module keeps them apart in memory as well (a list of texts, which is the only field that is preprocessed and indexed, and a metadata table keyed by doc_id).

Table 3. *Fields of the dataset.*

Field	Role	Example / note
doc_id	unique document ID	R0001
text	main text content (indexed)	raw review text with original markup and casing
category	class label	Appliances, Beauty, Musical Instruments, Pet Supplies, Software, Video Games
title	metadata - title	"I honestly don't know why I didn't get this sooner."
date	metadata - date	2000-06-13 to 2023-07-09
rating, helpful_votes, verified_purchase, has_images	metadata	1-5 stars; votes; True/False
asin, parent_asin, user_id	metadata - product and (anonymised) reviewer	used for the collaborative-filtering feasibility check
source	metadata - provenance	Amazon Reviews 2023 / Pet_Supplies.jsonl
dup_group, dup_type, is_injected	duplicate ground truth	used only to evaluate the duplicate detector

2.3 Corpus statistics

Table 4. *Corpus statistics shown on the Data Ingestion page after loading.*

Statistic	Value	Statistic	Value
Documents	1,224	Mean tokens / document	71.9
Categories	6	Median tokens / document	52
Raw text size	0.467 MB	Shortest / longest review	30 / 344 tokens
Raw tokens	88,004	Date range	2000-06-13 to 2023-07-09
Raw vocabulary (whitespace)	13,047	Metadata fields	14

2.4 Realistic noise

The reviews are not cleaned at source. Table 5 lists the noise that the preprocessing pipeline has to handle, measured on the raw text. For example, without markup removal the token "br" (from
) becomes a feature of 224 documents.

Table 5. *Noise profile of the raw text (share of documents containing each pattern).*

Noise pattern	Documents	Share
HTML markup (, tags)	224	18.3%
HTML entities (& ")	43	3.5%
ALL-CAPS words (3+ letters)	266	21.7%
repeated punctuation (!!, ??, ...)	202	16.5%
digits / model numbers	503	41.1%
non-ASCII (emoji, curly quotes)	199	16.3%
URLs	1	0.1%

2.5 Duplicate and near-duplicate documents

The corpus contains **60 ground-truth duplicate groups (121 documents, 9.9%)**. 36 groups occur naturally in the source data: the same reviewer posted the same text twice on the same day, usually for the same

product listing (2.5% of the sampled pool belongs to such groups; six groups per category were kept). To evaluate near-duplicate detection on known cases, 24 documented variants of other reviews were added (is_injected = True): re-posts with an ALL-CAPS first sentence and
 markup, re-posts with extra punctuation and emoticons, and edited re-posts with two words removed and an "UPDATE" sentence appended. Section 4.5 describes how the system detects them and Q2 (Section 9) how they affect the analyses.

2.6 Suitability for classification, clustering and recommendation

- **Classification and clustering evaluation:** six balanced, human-assigned category labels (201-207 documents each).
- **Content-based recommendation:** every review has enough text (30-344 tokens) to be compared with cosine similarity, plus metadata to display.
- **Collaborative recommendation:** user and product IDs exist, but only 62 of 1,105 users wrote more than one review and the user × item matrix is 0.10% dense, so this approach was not selected (Q3).

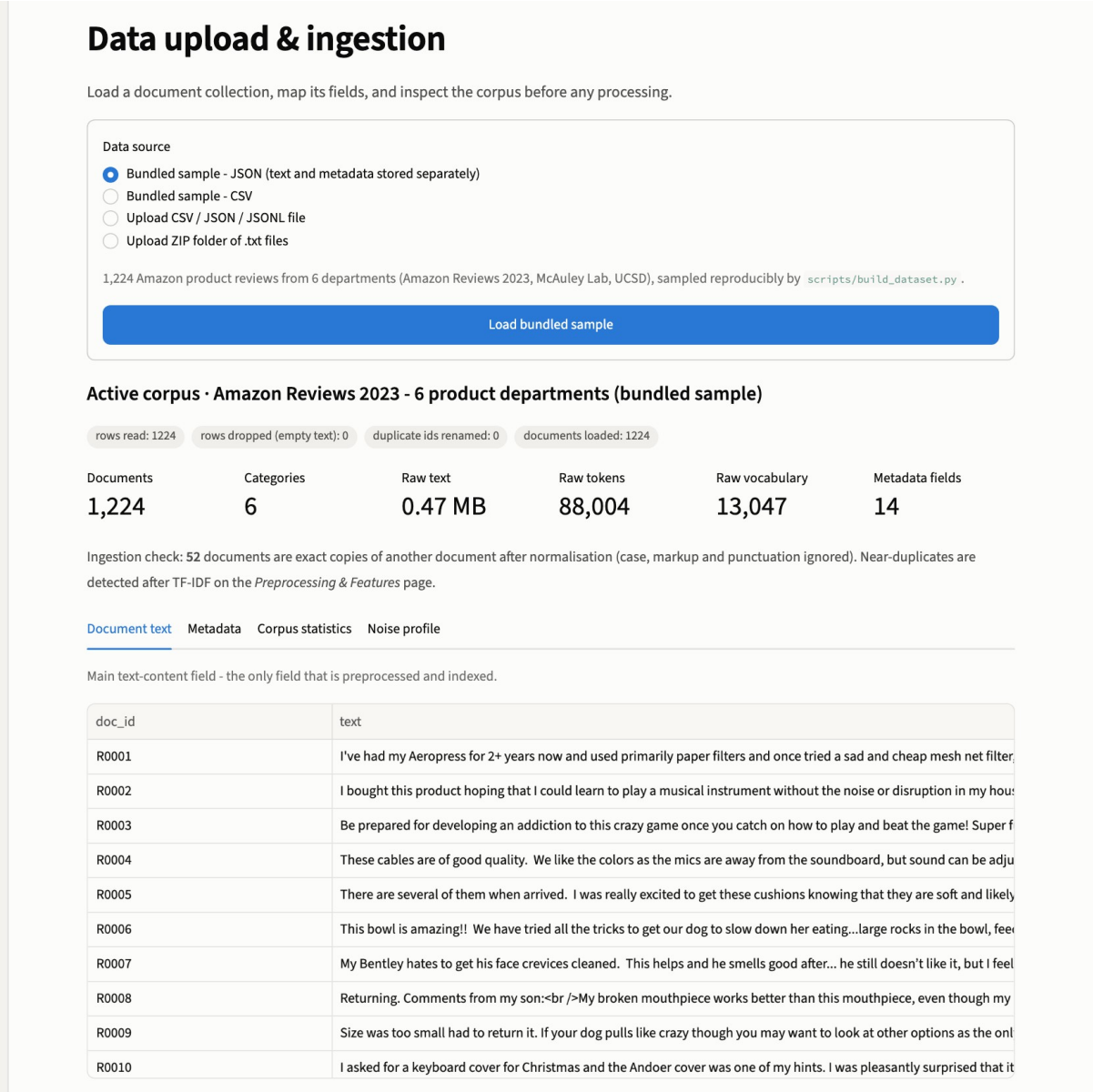


Figure 1. *Data Ingestion page: source selection (bundled JSON/CSV, CSV/JSON/JSONL upload, ZIP of .txt files), ingestion log, corpus statistics and the document-text table.*



Active corpus · Amazon Reviews 2023 - 6 product departments (bundled sample)

rows read: 1224 | rows dropped (empty text): 0 | duplicate ids renamed: 0 | documents loaded: 1224

Documents	Categories	Raw text	Raw tokens	Raw vocabulary	Metadata fields
1,224	6	0.47 MB	88,004	13,047	14

Ingestion check: **52** documents are exact copies of another document after normalisation (case, markup and punctuation ignored). Near-duplicates are detected after TF-IDF on the *Preprocessing & Features* page.

Document text | Metadata | Corpus statistics | Noise profile

Metadata kept in a separate table keyed by doc_id (never indexed).

doc_id	title	category	rating	date	helpful_votes	verified_pur
R0001	I honestly don't know why I didn't get this sooner.	Appliances	5	2017-06-03	2	☑
R0002	So much fun to use!	Musical Instruments	5	2020-03-05	5	☑
R0003	Super addictive!	Software	5	2017-11-25	87	☑
R0004	These cables are of good quality. We like the colors as the mics ...	Musical Instruments	5	2016-01-24	0	☑
R0005	It works well	Musical Instruments	4	2021-03-10	13	☑
R0006	Brake-Fast Bowl is a Godsend	Pet Supplies	5	2008-01-18	2	☐
R0007	Great for bulky face crevices	Pet Supplies	5	2021-04-27	3	☑
R0008	My broken mouthpiece works better than this mouthpiece	Musical Instruments	1	2018-03-05	0	☑

Metadata schema

field	role	dtype	non-empty	unique values	example
title	title	str	1224	1140	I honestly don't know why
category	category label	str	1224	6	Appliances
rating	metadata	float64	1224	5	5.0
date	metadata	str	1224	1017	2017-06-03
helpful_votes	metadata	int64	1224	45	2
verified_purchase	metadata	bool	1224	2	True
has_images	metadata	bool	1224	2	False
asin	metadata	str	1224	1083	B019HPS9OM
parent_asin	metadata	str	1224	1059	B07RDYVQXF
user_id	metadata	str	1224	1105	AGYULT5RFNZAYCUAQZDP

Figure 2. *Metadata tab: the metadata table keyed by doc_id and its schema, kept separate from the text.*

3. Implementation approach

3.1 Architecture

The system has three layers: a pure-Python library (textmining/) that contains every algorithm, a Streamlit front end (app.py and views/) that calls the library, and scripts that build the dataset and reproduce all report numbers. Because the app and scripts/run_experiments.py share the same functions and seeded defaults, the numbers in this report are exactly those shown by the app with the default configuration.

1. **Ingestion** (ingestion.py) - read CSV / JSON / JSONL / ZIP, map ID, text, label and title columns, validate (empty texts, duplicate IDs), split text from metadata.
2. **Preprocessing** (preprocessing.py) - markup and entity removal, URL removal, contraction expansion, lower-casing, regex tokenisation, stop words (negations kept), Porter stemming or WordNet lemmatisation, n-grams.

3. **Indexing and TF-IDF** (indexing.py) - inverted index with postings (doc, tf), df pruning, TF-IDF weights read from the postings, cosine length normalisation, term-at-a-time cosine scoring.
4. **Duplicates** (duplicates.py) - normalised-text hashing, cosine threshold, union-find groups, keep-one policy.
5. **Text mining** - Multinomial Naive Bayes (classification.py), K-Means (clustering.py), Purity and Rand Index (evaluation.py), analyst insights (analytics.py).
6. **Recommendation** (recommendation.py) - Top-K cosine recommender with duplicate collapsing and score explanations; P@K evaluation.
7. **Experiments** (experiments.py, scripts/run_experiments.py) - preprocessing ablation, duplicate study, K-Means variants, k-sensitivity, detection sweep.

3.2 Streamlit workflow

Table 6. *Pages of the Streamlit application (sidebar navigation).*

Page	Contents
Dashboard	business context, KPIs, headline results with chance levels, category / year / rating / length charts, workflow
Data Ingestion	bundled data or upload (CSV, JSON, JSONL, ZIP of .txt), column mapping, separate text and metadata tables, corpus statistics, noise profile, export
Preprocessing & Features	pipeline configuration form; tabs: pipeline preview, vocabulary and term frequencies before/after, inverted-index explorer and TF-IDF vector inspector, cosine similarity, duplicates
Text Mining Hub	Naive Bayes (metrics, actual vs predicted, confusion matrix, predictions, discriminative terms, try-it box); K-Means (seeds, iterations, RSS, stopping, 2-D scatter, sizes, membership, top terms); evaluation and comparison; corpus insights
Recommendations	select a review or type text; Top-K with scores, metadata and shared terms; P@K evaluation; method and limitations
Performance Analytics	scorecard, run times, preprocessing ablation (Q1), duplicate study (Q2), K-Means variants and k-sensitivity (Q5)

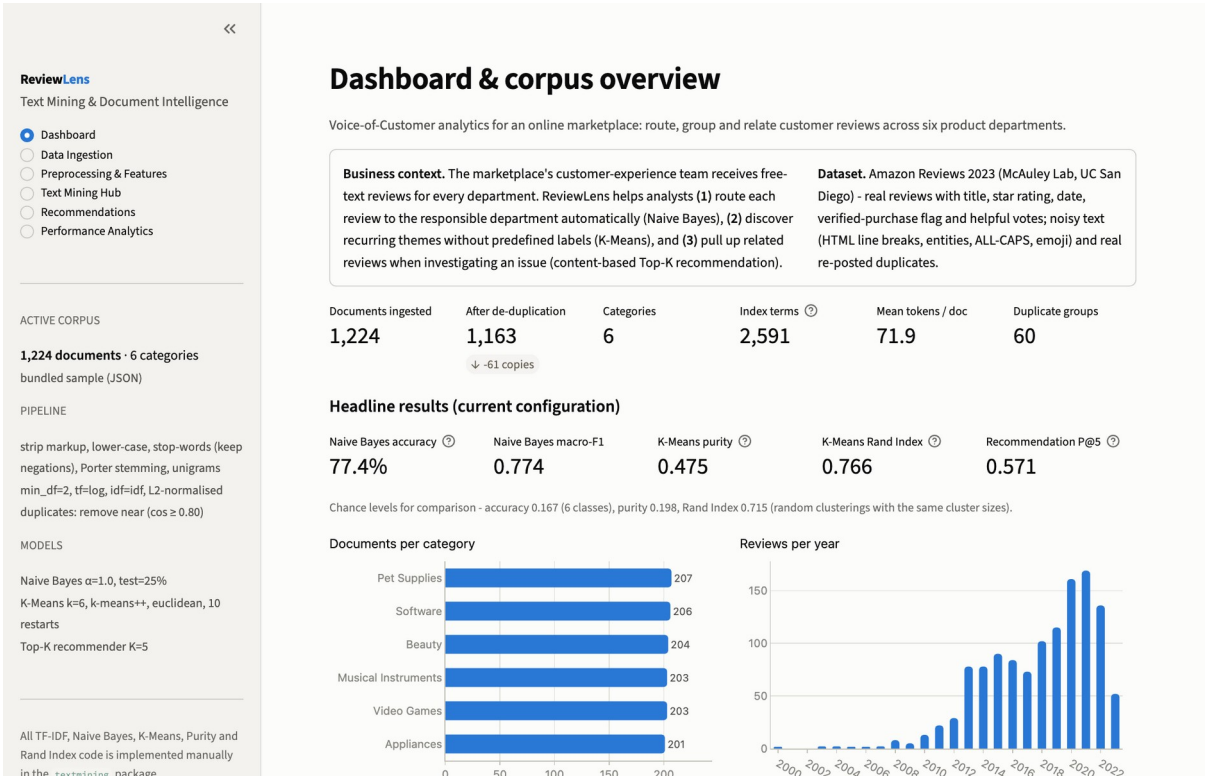


Figure 3. *The application running locally (Dashboard page with sidebar navigation and active configuration).*

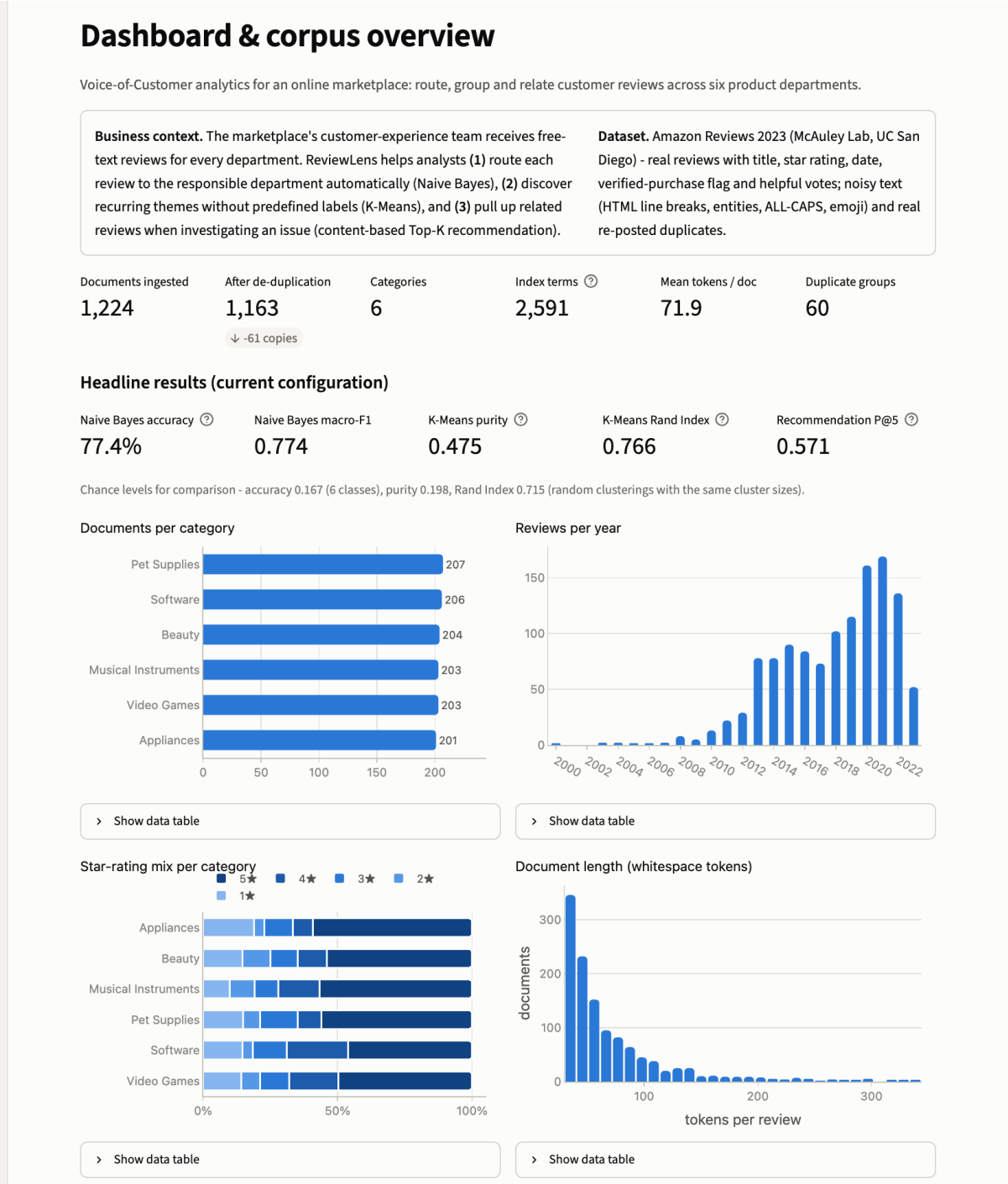


Figure 4. *Dashboard and corpus overview: KPIs, headline results and corpus charts.*

3.3 Manually implemented algorithms

The brief prohibits library implementations of TF-IDF, Naive Bayes, K-Means, Purity and Rand Index. None is used - scikit-learn is not imported anywhere. numpy is used only for vector arithmetic (dot products, means, the eigen-decomposition behind the 2-D plot) and NLTK only for the Porter stemmer and the WordNet lemmatiser.

Table 7. *Where each required technique is implemented.*

Requirement	Implementation (file · function)
Tokenisation, stop words, n-grams	preprocessing.py · tokenize, stopword_set, make_ngrams, Preprocessor
Inverted index	indexing.py · InvertedIndex (term → postings [(doc, tf)], df, cf, pruning)
TF-IDF with length normalisation	indexing.py · tf_weight, idf_weight, TfidfModel (weights from postings, L2 normalisation)
Cosine similarity	indexing.py · cosine_similarity, TfidfModel.score (term-at-a-time), similarity_matrix
Naive Bayes, split, metrics	classification.py · MultinomialNaiveBayes, stratified_split, classification_report, confusion_matrix
K-Means	clustering.py · initial_seeds (random / k-means++), kmeans_single (assign, update, stop), kmeans (restarts)
Purity, Rand Index	evaluation.py · purity, pair_counts_bruteforce (definition), pair_counts (contingency), rand_index
Recommendation	recommendation.py · ContentRecommender, evaluate_precision_at_k

3.4 Verification and reproducibility

tests/test_core.py checks the implementations against the worked examples of Introduction to Information Retrieval (12/12 tests passed). Table 8 lists the textbook cases.

Table 8. *Correctness tests against textbook examples (Manning, Raghavan & Schütze, 2008).*

Test	Expected (book)	Result
Naive Bayes, Example 13.1 (Chinese/Tokyo/Japan)	score(China) 0.0003 > score(not China) 0.0001, so class China	pass
Purity, Figure 16.4 (17 points, 3 clusters)	12/17 = 0.71	pass
Rand Index, Figure 16.4	TP 20, FP 20, FN 24, TN 72; RI = 0.68	pass (both implementations)
TF-IDF and cosine, Section 6.4 worked example (best car insurance, lnc.ltc)	normalised weights 0.34/0.52/0.78; score 0.80	pass
Inverted index scoring vs matrix cosine	identical scores	pass
K-Means, two separated groups	recovers both groups; RSS never increases	pass

Reproducibility: all randomness uses seeded Fisher-Yates shuffles driven by random.Random(seed). Vector lengths use math.fsum (exactly rounded), K-Means treats distances within 1e-9 as ties (lowest cluster index wins) and rankings use similarities rounded to 12 decimals. All 23 result files and every value in results/summary.json were verified to agree to 9 decimal places on Python 3.9 / numpy 1.24 / Streamlit 1.30 and on Python 3.14.6 / numpy 2.5.4 / Streamlit 1.67.0. The only exception is the optional lemmatisation row of the ablation (Table 31), whose output depends on the installed NLTK/WordNet version.

As an extra check outside the submitted code, results were compared in a separate throwaway environment. scikit-learn's Multinomial Naive Bayes predicted the same label for all 288 test reviews; its accuracy, precision, recall, F1, confusion matrix, Purity (from its contingency matrix), Rand Index and cosine values match ours; and the TF-IDF weights match an independent numpy recomputation. K-Means from the same seeds reached a different local optimum (RSS 1123.089 vs 1123.000), because 10 documents share no term with any seed and are exact ties in the first iteration, which the two programs break differently. Started from our final centroids, scikit-learn's K-Means keeps the same partition, and on tie-free data the two give identical results, which confirms that ours is a valid K-Means solution.

4. Preprocessing and feature choices

4.1 Configurable pipeline

Every step can be switched on or off on the Preprocessing & Features page (Figure 5); the defaults used in this report are listed in Table 9 together with the reason for choosing them.

Table 9. *Default preprocessing and feature settings.*

Step	Default	Reason
Markup and entities	removed (, tags, &)	"br" would otherwise be a feature of 224 documents
URLs / e-mail	removed	no topical meaning
Contractions	expanded (didn't → did not)	keeps the negation as its own token
Lower-casing	on	merges GREAT / Great / great
Tokenisation	regex: runs of letters/digits, min. length 2, pure numbers dropped	removes punctuation and emoji; keeps model names such as ps4
Stop words	English list + review boilerplate (product, item, amazon, bought …), negations kept	removes half of all tokens; not/no/never carry sentiment
Normalisation	Porter stemming (WordNet lemmatisation optional)	best NB and recommendation results with the smallest vocabulary (Q1)
N-grams	unigrams (bigrams / trigrams optional)	n-grams lowered every score on these short reviews (Q1)
Vocabulary pruning	min_df = 2, max_df = 0.95, cap 5,000	drops misspellings and one-off terms
TF-IDF	log tf: 1 + log10(tf); idf = log10(N/df); cosine normalisation	SMART ltc weighting (IIR ch. 6)
Duplicates	remove exact + near-duplicates (cosine ≥ 0.80), keep first	prevents leakage and copy-filled recommendation lists (Q2)

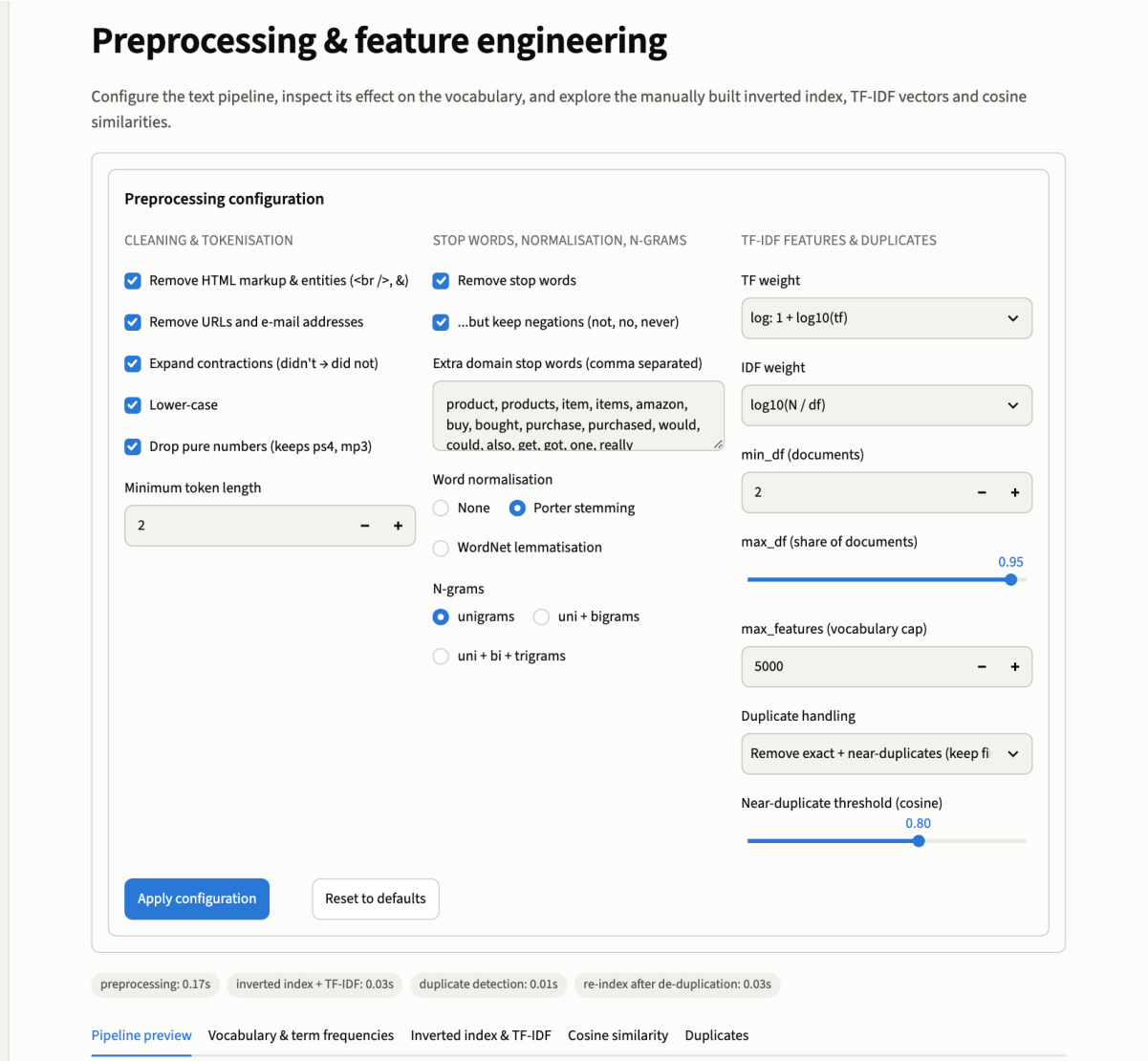


Figure 5. *Preprocessing configuration form and run times of the pipeline stages.*

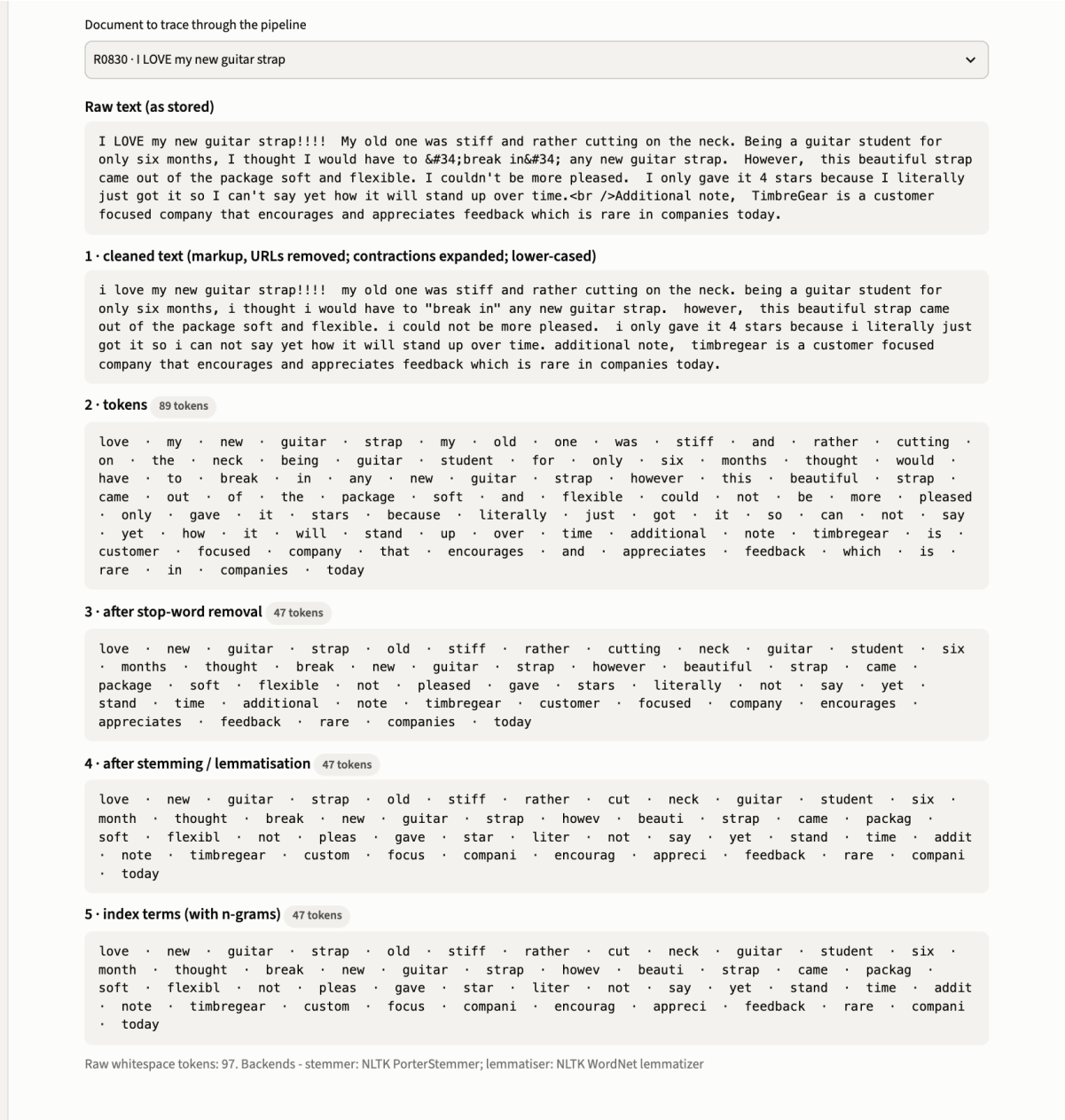


Figure 6. *Pipeline preview: one noisy review traced through every step (raw → cleaned → tokens → stop words removed → stems → index terms).*

4.2 Effect on vocabulary and term frequencies

Table 10. *Vocabulary after each cumulative preprocessing step (analysis corpus).*

Stage	Vocabulary	Tokens	Type/token	Hapax
0. raw whitespace tokens	13,000	84,357	0.1541	8,112
1. cleaned + tokenised (case kept)	8,328	79,295	0.105	4,184
2. + lower-casing	7,158	79,295	0.0903	3,382
3. + stop-word removal	7,021	39,556	0.1775	3,378
4. + Porter stemming	4,987	39,556	0.1261	2,148
5. + n-grams (1-1)	4,987	39,556	0.1261	2,148

Before preprocessing the most frequent tokens are function words ("the", "I", "and", "to", "a") and the same word appears in several surface forms ("The" and "the" are different tokens). After preprocessing

the top terms are content words ("use", "game", "work", "great", "love"), and "not" is the most frequent term because negations are deliberately kept. After df pruning 2,591 index terms remain.

Table 11. *Top-15 tokens before and terms after preprocessing.*

Rank	Raw token	Frequency	Processed term	Frequency
1	the	3,427	not	1,339
2	I	2,546	use	529
3	and	2,287	game	410
4	to	2,247	like	407
5	a	2,054	work	401
6	it	1,574	great	332
7	is	1,322	good	282
8	for	1,070	love	261
9	of	1,069	time	232
10	this	897	play	228
11	my	839	look	218
12	in	761	well	201
13	that	691	no	196
14	have	687	make	192
15	on	665	need	187

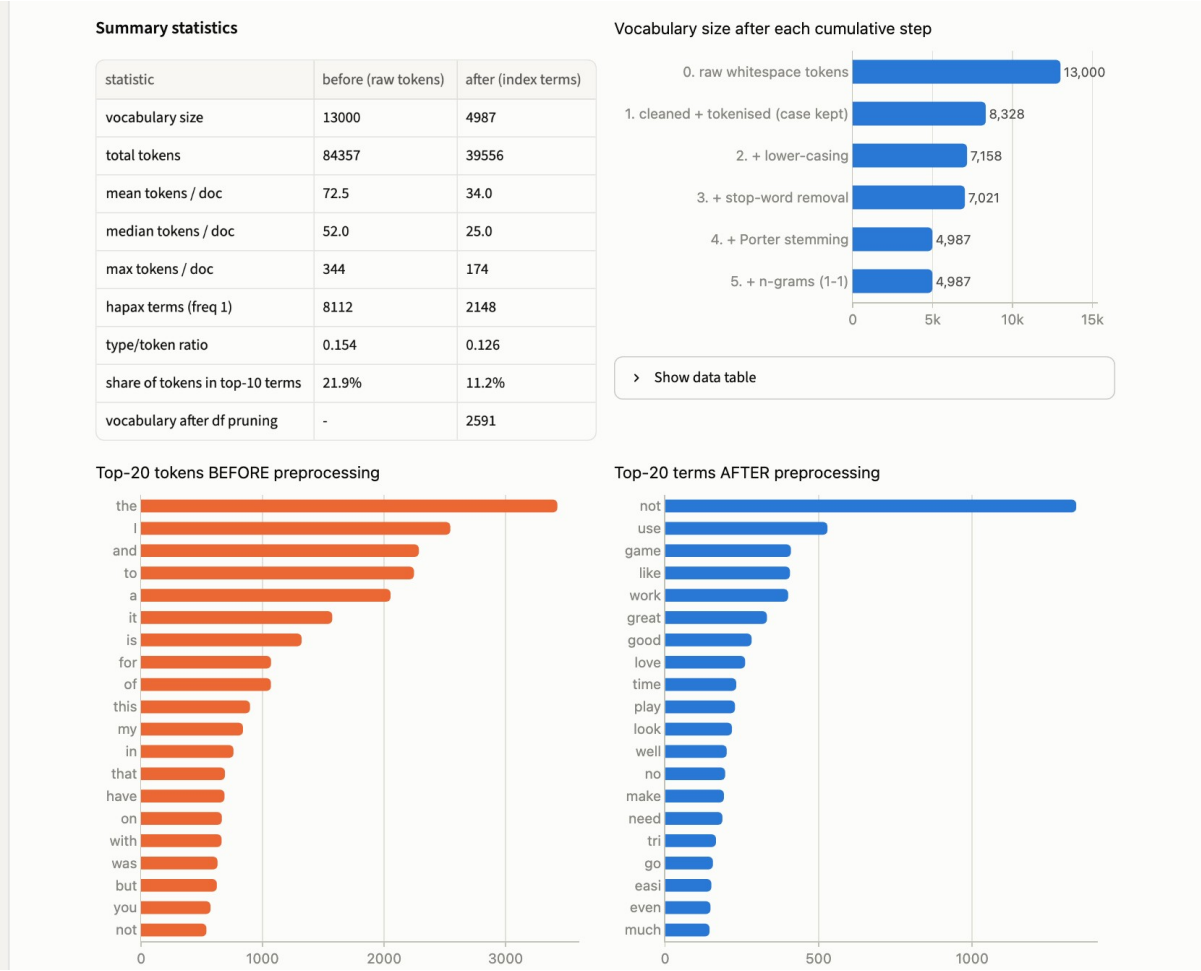


Figure 7. *Vocabulary and term frequencies before vs after preprocessing (1/2): summary statistics, vocabulary after each step and the top-20 tokens / terms.*

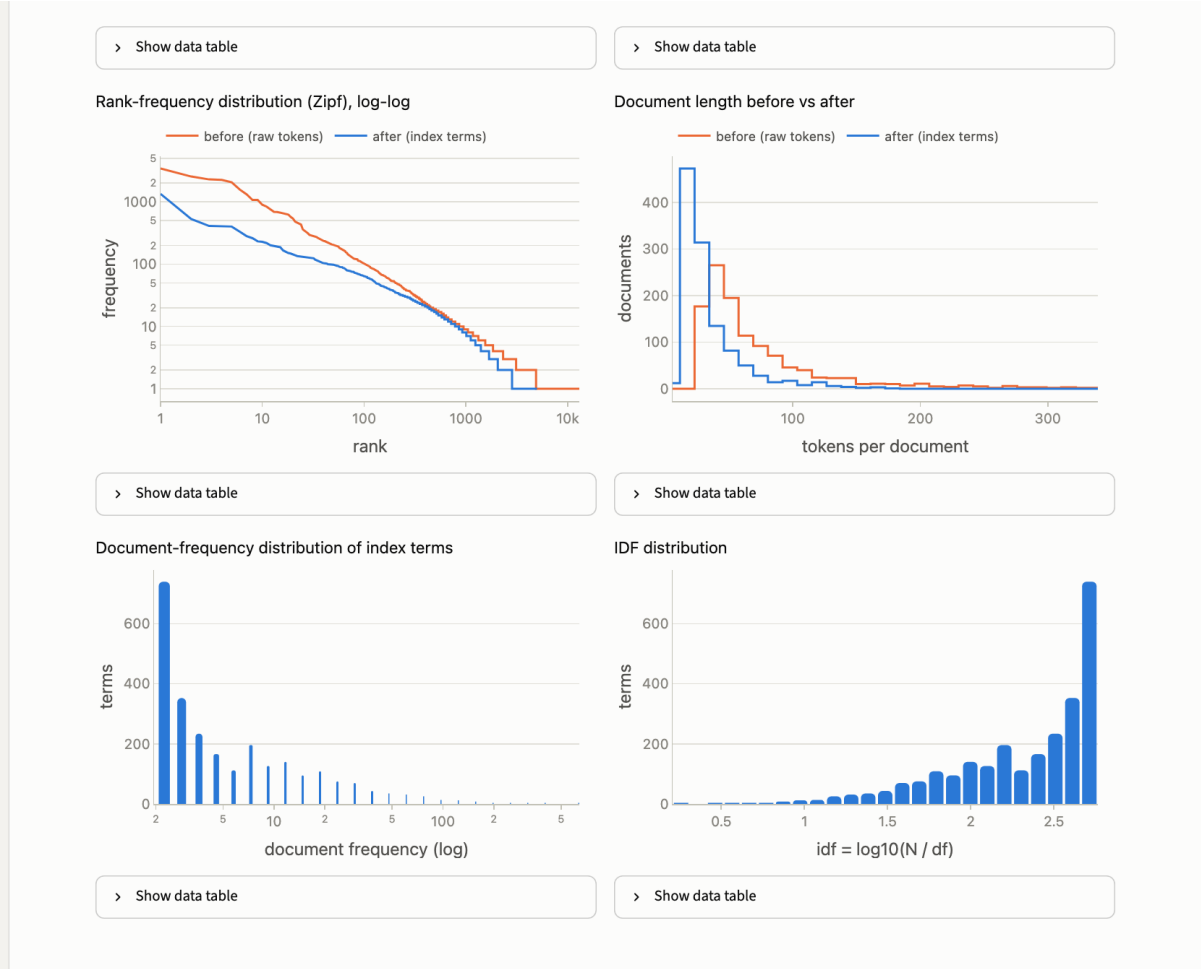


Figure 8. *Vocabulary and term frequencies before vs after preprocessing (2/2): rank-frequency (Zipf) plot, document lengths, document-frequency and idf distributions.*

4.3 Inverted index and TF-IDF (manual)

The inverted index maps every term to its postings list of (document, term frequency) pairs. TF-IDF weights are computed by walking each postings list, then every document vector is divided by its Euclidean length, so all vectors have unit length:

$$w(t,d) = (1 + \log10\ tf(t,d)) \times \log10(N / df(t))$$

$$\hat{w}(t,d) = w(t,d) / sqrt(\ \Sigma_t'\ w(t',d)^2\ )$$

For the analysis corpus the index holds 2,591 terms and 30,356 postings; the document-term matrix is 1,163 × 2,591 with 1.01% non-zero cells. The app's index explorer shows df, idf and the full postings list of any term (e.g. "batteries" → stem "batteri", df 22, idf 1.7232) and the document-vector inspector shows tf, idf, raw and normalised weights of a review, confirming $\Sigma\ \hat{w}^2 = 1$.

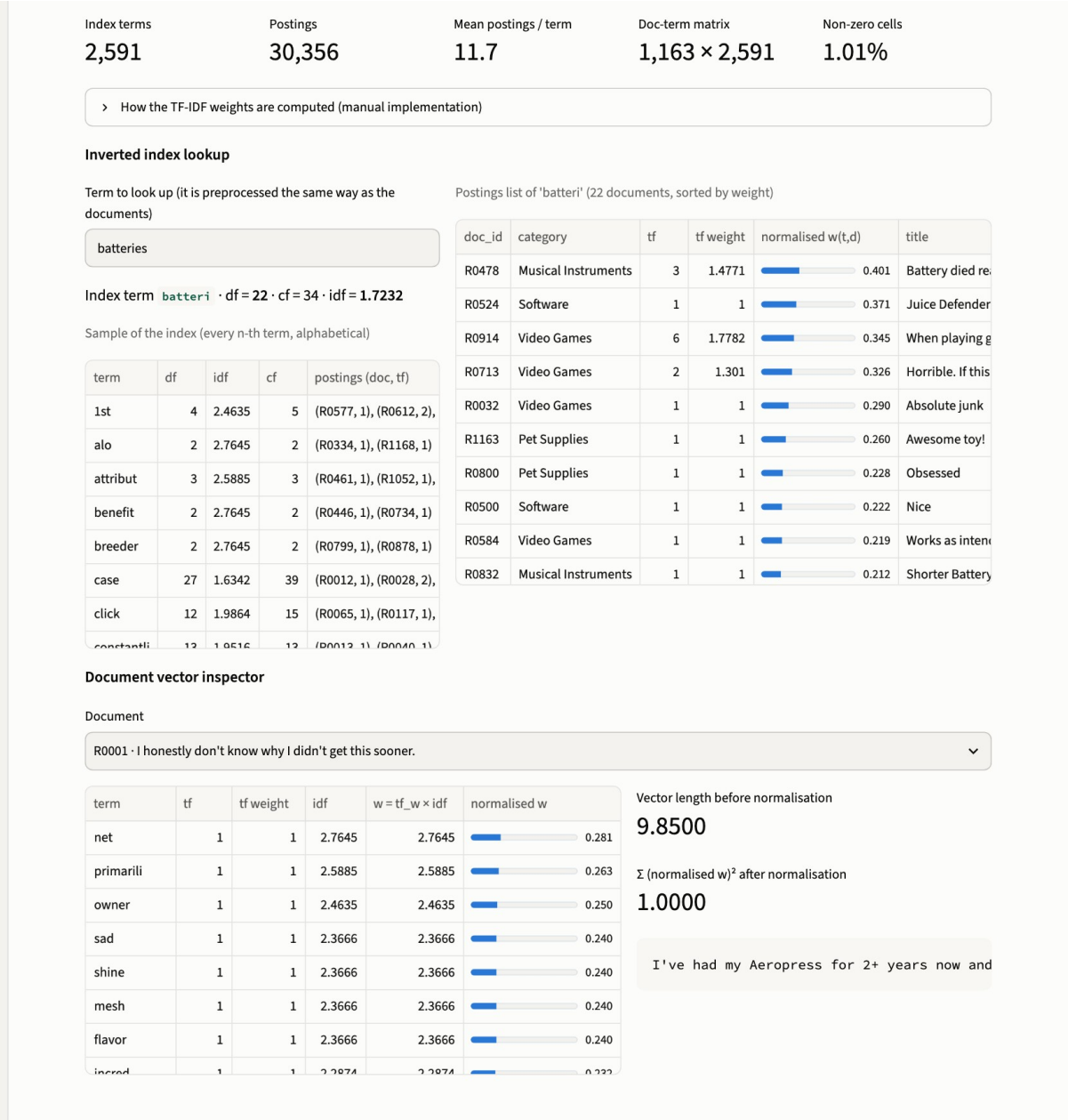


Figure 9. *Inverted index explorer (df, idf, postings with normalised weights) and TF-IDF document-vector inspector.*

4.4 Document similarity

$$\cos(d1, d2) = \Sigma_t\ \hat{w}(t,d1) \cdot \hat{w}(t,d2) \qquad \text{(vectors have unit length)}$$

For a single query the recommender accumulates this sum term-at-a-time through the inverted index (IIR fig. 6.14), so only documents that share a term with the query are touched; for all-pairs analyses the same values are computed as $X \cdot X^{T}$. Pairs from the same category are more similar on average than pairs from different categories (mean cosine 0.029 vs 0.018); both are low because short reviews share few terms.

4.5 Duplicate detection and handling

Exact duplicates are found with a hash table of the normalised text (lower-cased, markup and punctuation removed); near-duplicates are pairs with TF-IDF cosine ≥ 0.80; pairs are merged into groups with union-find. On the full corpus the detector finds 60 groups (51 exact-copy groups, 62 near-duplicate pairs) and recovers **62 of 62 ground-truth pairs with no false positive (precision 1.000, recall 1.000)**. The least similar true duplicate pair has cosine 0.886 while the most similar non-duplicate pair has 0.496;

Table 12 shows how recall falls when the threshold is set too high. By default one document per group is kept (61 documents removed, 1,163 analysed).

Table 12. *Near-duplicate detector vs ground truth for different cosine thresholds.*

Threshold	Detected pairs	Correct pairs	Precision	Recall	F1
0.5	62	62	1.000	1.000	1.000
0.6	62	62	1.000	1.000	1.000
0.7	62	62	1.000	1.000	1.000
0.8	62	62	1.000	1.000	1.000
0.85	62	62	1.000	1.000	1.000
0.9	61	61	1.000	0.984	0.992
0.95	57	57	1.000	0.919	0.958
0.99	54	54	1.000	0.871	0.931

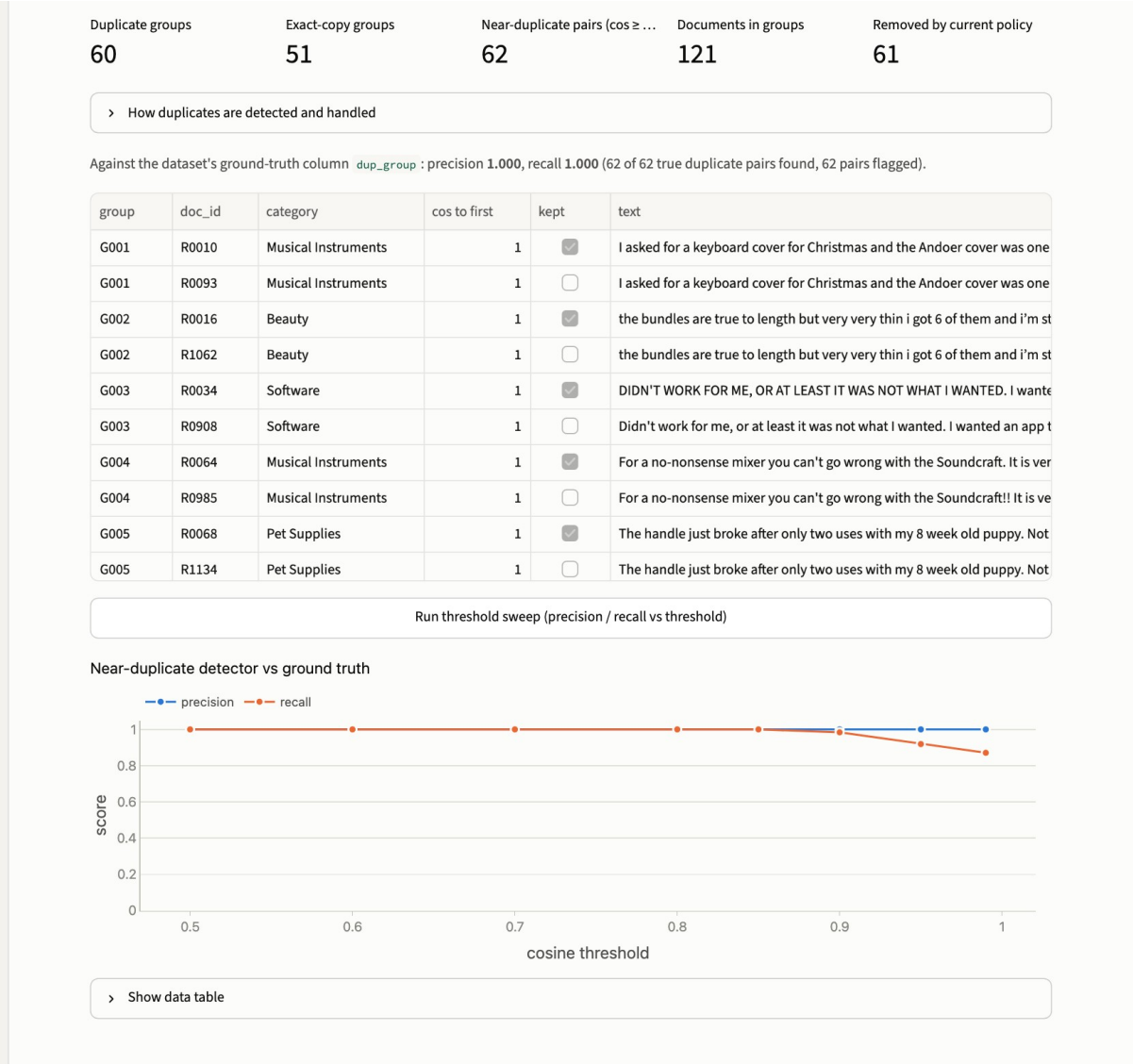


Figure 10. *Duplicates tab: detected groups (with the copy that is removed), evaluation against ground truth and the threshold sweep.*

5. Classification results (Naive Bayes)

5.1 Method and set-up

A Multinomial Naive Bayes classifier was implemented from scratch. It is trained only on the training split: class priors and term likelihoods with add-one (Laplace) smoothing over the training vocabulary; prediction sums log-probabilities and ignores terms not seen in training.

$$P(c) = N_c / N \qquad P(t|c) = (T_{ct} + \alpha) / (\Sigma_{t'} T_{ct'} + \alpha|V|), \quad \alpha = 1$$

$$c_{map} = \text{argmax}_c \left[\log P(c) + \Sigma_{t \in d} tf(t,d) \cdot \log P(t|c) \right]$$

The 1,163 de-duplicated reviews are split 75/25, stratified by category and duplicate-aware (seed 537): 875 training and 288 test reviews.

5.2 Evaluation metrics

Table 13. *Naive Bayes results on the test set (default split) and over 10 random splits.*

Metric	Default split	Mean ± sd over 10 splits
Accuracy	0.774	0.782 ± 0.024
Macro precision	0.774	
Macro recall	0.774	
Macro F1	0.774	0.783 ± 0.024
Weighted F1	0.774	
Chance level (majority class of the test set)	0.167	

Table 14. *Per-class precision, recall and F1 (default split).*

Class	Precision	Recall	F1	Support
Appliances	0.860	0.896	0.878	48
Beauty	0.778	0.729	0.753	48
Musical Instruments	0.714	0.729	0.722	48
Pet Supplies	0.833	0.833	0.833	48
Software	0.787	0.771	0.779	48
Video Games	0.673	0.688	0.680	48

Table 15. *Confusion matrix (rows = actual, columns = predicted).*

Actual \ predicted	Appliances	Beauty	Musical Instr.	Pet Supplies	Software	Video Games
Appliances	43	2	2	1	0	0
Beauty	1	35	6	3	1	2
Musical Instruments	2	1	35	2	3	5
Pet Supplies	2	4	2	40	0	0
Software	1	0	0	1	37	9
Video Games	1	3	4	1	6	33

Accuracy is 4.6× the majority-class rate. The best class is Appliances (F1 0.878), whose vocabulary is very specific (coffee, filter, refrigerator, ice); the weakest is Video Games (F1 0.680). The largest single confusion is between Software and Video Games (9 Software reviews predicted as Video Games and 6 the other way round, 15 of the 65 errors): the Software department contains many app-store games, so both use words such as game, play and fun. Next come Musical Instruments / Video Games (9 errors),

and Beauty with Pet Supplies and with Musical Instruments (7 and 7). Several of the misrouted reviews are short on department vocabulary, with generic titles such as "Works well" or "Great find!!".

5.3 Actual and predicted labels

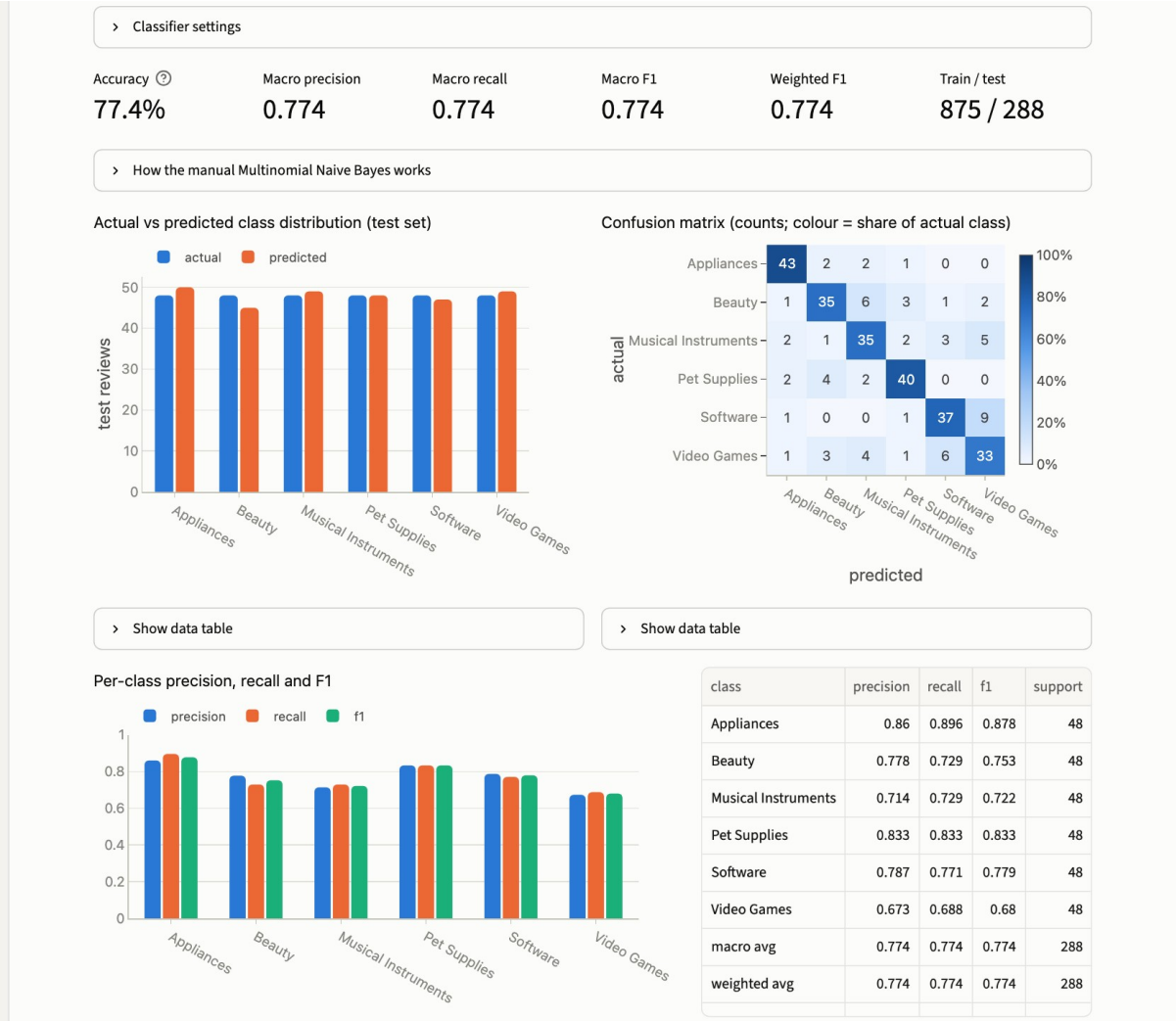
Table 16 shows an excerpt of the 288 test predictions (including errors, highlighted); the complete list with posterior confidence is shown in the app (Figure 12) and saved in results/nb_predictions.csv. 65 test reviews are misclassified.

Table 16. *Excerpt of actual vs predicted labels (errors highlighted).*

doc_id	Title	Actual	Predicted	Confidence	Correct
R0003	Super addictive!	Software	Video Games	0.546	NO
R0005	It works well	Musical Instruments	Musical Instruments	0.951	yes
R0008	My broken mouthpiece works better than this …	Musical Instruments	Musical Instruments	1.000	yes
R0017	This game has become very addictive I just w…	Software	Software	0.731	yes
R0020	My dog loves the feel of the brush	Pet Supplies	Beauty	1.000	NO
R0021	Awesome	Appliances	Appliances	1.000	yes
R0024	Don’t buy if your in the US!!!	Video Games	Musical Instruments	0.821	NO
R0025	Ok	Video Games	Video Games	0.983	yes
R0029	It works on Canine Warts!	Pet Supplies	Pet Supplies	1.000	yes
R0032	Absolute junk	Video Games	Video Games	0.914	yes
R0033	Amazing product!	Beauty	Beauty	1.000	yes
R0056	You too can feel like Lawrence Welk!	Musical Instruments	Pet Supplies	0.983	NO
R0071	Not happy with this purchase	Video Games	Software	0.995	NO
R0087	FALSE ADVERTISING	Beauty	Video Games	0.880	NO

Table 17. *Most discriminative (stemmed) terms per class: log P(t|c) minus the mean over the other classes.*

Class	Top terms
Appliances	ice, coffe, filter, refriger, cup, maker, dryer, foil, stove, water
Beauty	hair, skin, brush, bottl, oil, wig, curl, smell, blond, polish
Musical Instruments	guitar, amp, string, pedal, sound, instrument, mic, bass, speaker, karaok
Pet Supplies	dog, cat, puppi, litter, har, food, toy, eat, collar, cage
Software	app, game, kindl, download, program, tax, account, version, fire, fun
Video Games	game, headset, keyboard, mous, charact, consol, xbox, player, control, ear



class	precision	recall	f1	support
Appliances	0.86	0.896	0.878	48
Beauty	0.778	0.729	0.753	48
Musical Instruments	0.714	0.729	0.722	48
Pet Supplies	0.833	0.833	0.833	48
Software	0.787	0.771	0.779	48
Video Games	0.673	0.688	0.68	48
macro avg	0.774	0.774	0.774	288
weighted avg	0.774	0.774	0.774	288

Figure 11. *Naive Bayes in the Text Mining Hub: metrics, actual vs predicted class distribution, confusion matrix and per-class precision / recall / F1.*

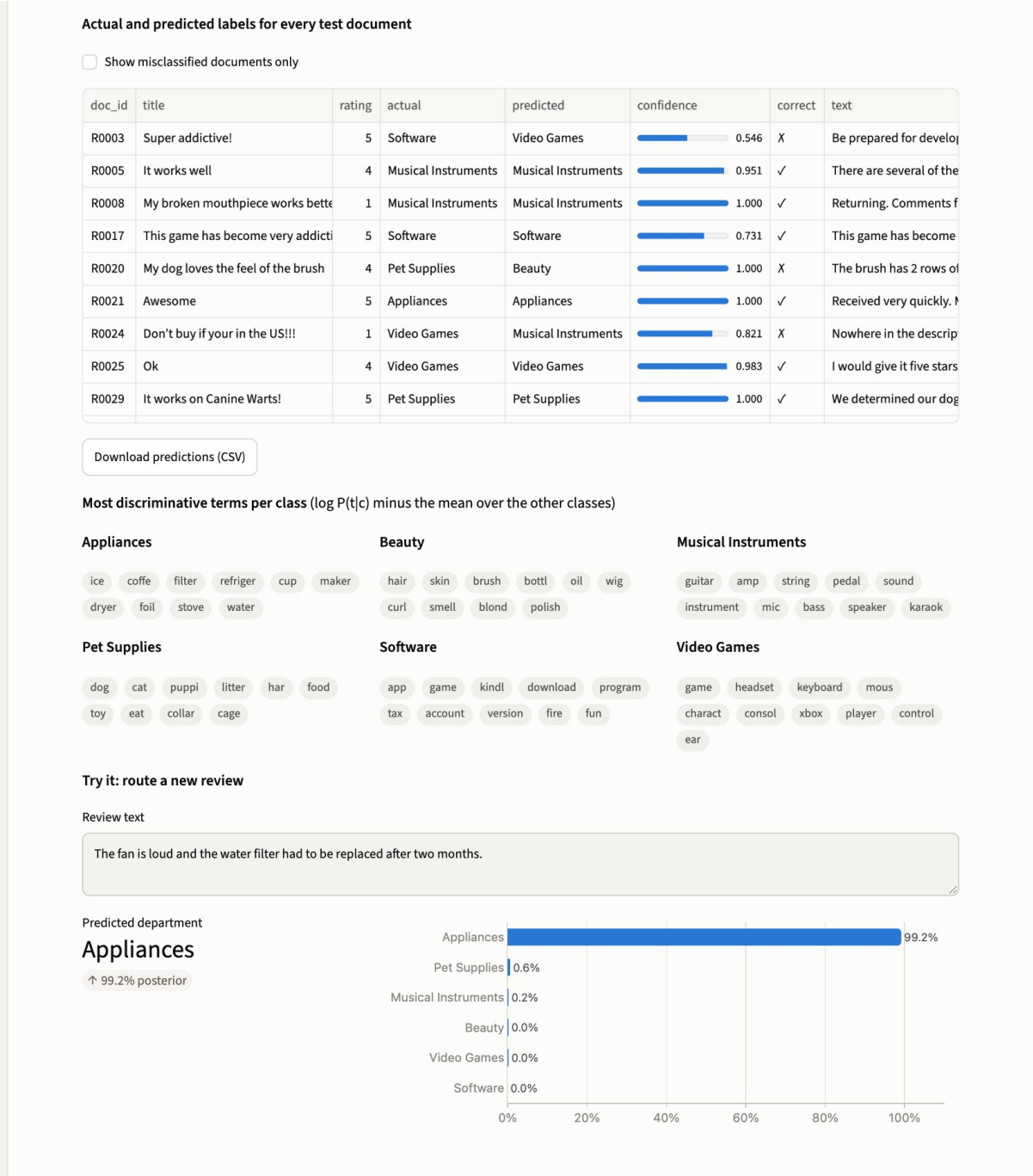


Figure 12. *Actual and predicted labels for every test document, discriminative terms and the 'route a new review' box.*

6. Clustering results (K-Means)

6.1 Algorithm and its steps

K-Means is implemented from scratch on the unit-length TF-IDF vectors (k = 6, the number of departments). Each run shows the four steps of the algorithm in the app (Figure 13):

- **Centroid initialisation** - k-means++: the first seed is a random document; each next seed is drawn with probability proportional to its squared distance from the nearest chosen seed.
- **Document assignment** - every document joins the centroid with the smallest squared Euclidean distance $|x - \mu|^2$.

- **Centroid update** - every centroid becomes the mean of its documents (an empty cluster is re-seeded with the farthest document).
- **Stopping criterion** - stop when no document changes cluster, when the largest centroid shift is below 10^{-4}, or after 100 iterations. The objective RSS = $\Sigma_k \Sigma_{x \in \omega_k} |x - \mu_k|^2$ never increases. Ten restarts (seeds 537-546) are run and the lowest-RSS run is kept.

Table 18. *Step 1 - seed documents of the best run (seed 537, k-means++).*

Cluster	Seed doc_id	Category	Title
0	R0871	Beauty	Excellent product
1	R1216	Video Games	Dials are useless
2	R0263	Pet Supplies	My Cat Loves It!
3	R0543	Video Games	Great quality! Great color! Great look
4	R0857	Appliances	Fixed my problem
5	R0162	Musical Instruments	For what you're paying, this is a good value.

Table 19. *Steps 2-4 - assignment and update iterations of the best run (first and last iterations).*

Iteration	Docs reassigned	RSS	Max centroid shift	Cluster sizes
1	1163	1130.384	0.9583	164, 181, 209, 121, 286, 202
2	82	1129.185	0.0326	162, 193, 207, 126, 266, 209
3	55	1128.493	0.0279	161, 195, 206, 126, 261, 214
4	29	1128.162	0.0193	160, 194, 205, 125, 267, 212
…	…			
27	1	1123.000	0.0074	136, 178, 194, 134, 277, 244
28	0	1123.000	0.0000	136, 178, 194, 134, 277, 244

In iteration 1 all 1,163 documents are assigned; the number of moves then drops quickly and the run stops with "converged: no document changed cluster in iteration 28". The final RSS is 1123.000; across the ten restarts RSS ranged from 1123.000 to 1125.114.

6.2 Cluster assignments

Table 20. *Excerpt of cluster assignments (complete table in the app and results/cluster_assignments.csv).*

doc_id	Title	True label	Cluster
R0001	I honestly don't know why I didn't get this sooner.	Appliances	C4
R0002	So much fun to use!	Musical Instruments	C1
R0003	Super addictive!	Software	C1
R0004	These cables are of good quality. We like the colors as the…	Musical Instruments	C5
R0005	It works well	Musical Instruments	C5
R0006	Brake-Fast Bowl is a Godsend	Pet Supplies	C2
R0007	Great for bulky face crevices	Pet Supplies	C5
R0008	My broken mouthpiece works better than this mouthpiece	Musical Instruments	C5
R0009	Order wrong size but after returning, I ordered a different…	Pet Supplies	C2
R0010	Covers well and keeps dust off	Musical Instruments	C5
R0011	It rocks	Video Games	C1
R0012	A cute little puzzle game	Software	C1

Table 21. *Clusters: size, majority category and highest-weighted centroid terms.*

Cluster	Size	Majority category	Share	Top terms (stemmed)
C0	136	Beauty	0.640	skin, hair, brush, color, use, dri, look, soft, nice, appli
C1	178	Software	0.579	game, play, fun, app, love, enjoy, time, like, free, kid
C2	194	Pet Supplies	0.562	cat, dog, water, love, easi, filter, like, toy, littl, keep
C3	134	Beauty	0.254	keyboard, hair, use, color, love, mani, even, sturdi, went, like
C4	277	Appliances	0.375	work, replac, great, instal, use, new, year, make, tri, not
C5	244	Musical Instruments	0.475	qualiti, good, sound, guitar, price, well, like, better, look, nice

Table 22. *Contingency table: clusters × true categories.*

Cluster	Appliances	Beauty	Musical Instr.	Pet Supplies	Software	Video Games
C0	9	87	7	16	7	10
C1	0	0	5	0	103	70
C2	41	17	11	109	11	5
C3	12	34	15	26	15	32
C4	104	27	40	22	49	35
C5	28	29	116	21	8	42

6.3 Purity and Rand Index

Both metrics are implemented manually (evaluation.py). Purity credits each cluster with its most frequent true category; the Rand Index counts, over all N(N−1)/2 document pairs, how often the clustering and the labels agree. The Rand Index is computed twice - by a literal pair-by-pair loop and from the contingency table - and both give identical counts.

$$\text{purity}(\Omega, C) = (1/N)\ \Sigma_k \max_j |\omega_k \cap c_j| \qquad RI = (TP + TN) / (TP + FP + FN + TN)$$

Table 23. *K-Means evaluation (default run, k = 6) with chance baselines and variability.*

Metric	Default run	Chance level	Mean ± sd over 10 seeds (each best of 10 restarts)
Purity	0.475	0.198	0.469 ± 0.044
Rand Index	0.766	0.715	0.754 ± 0.016
TP / FP pairs	37,324 / 83,113		
FN / TN pairs	74,809 / 480,457		

The chance level is obtained by shuffling the cluster ids across documents (same cluster sizes, 20 rounds). Purity is 2.4× the chance level. The Rand Index of 0.766 looks high, but a random clustering of the same sizes already scores 0.715, because 71.1% of all pairs are "different category, different cluster" (TN). Purity and the chance comparison are therefore the more informative numbers here.

6.4 Visualisations

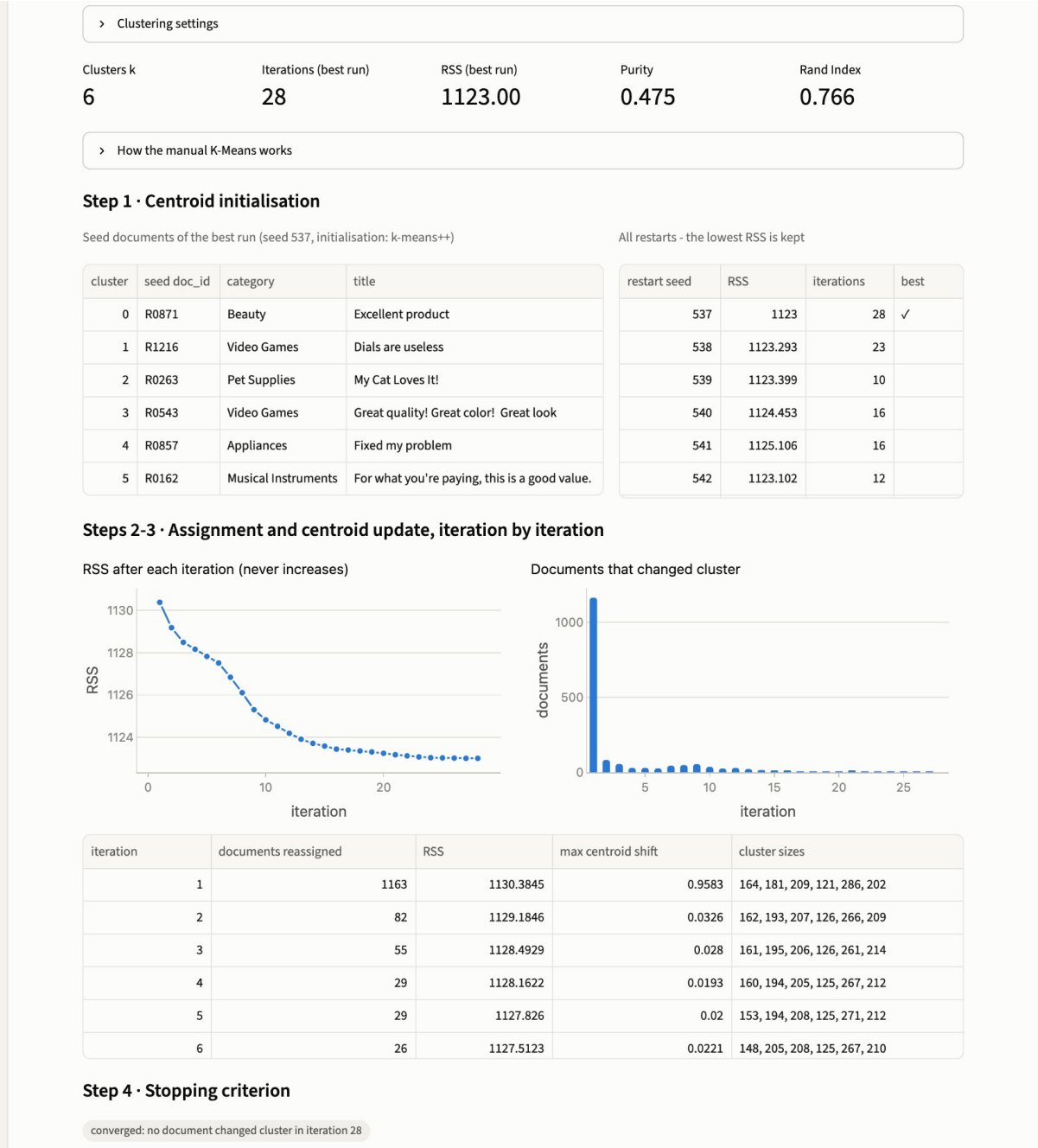


Figure 13. *K-Means steps in the app: seed documents of the best run, all restarts, RSS and document moves per iteration, iteration log and the stopping criterion.*

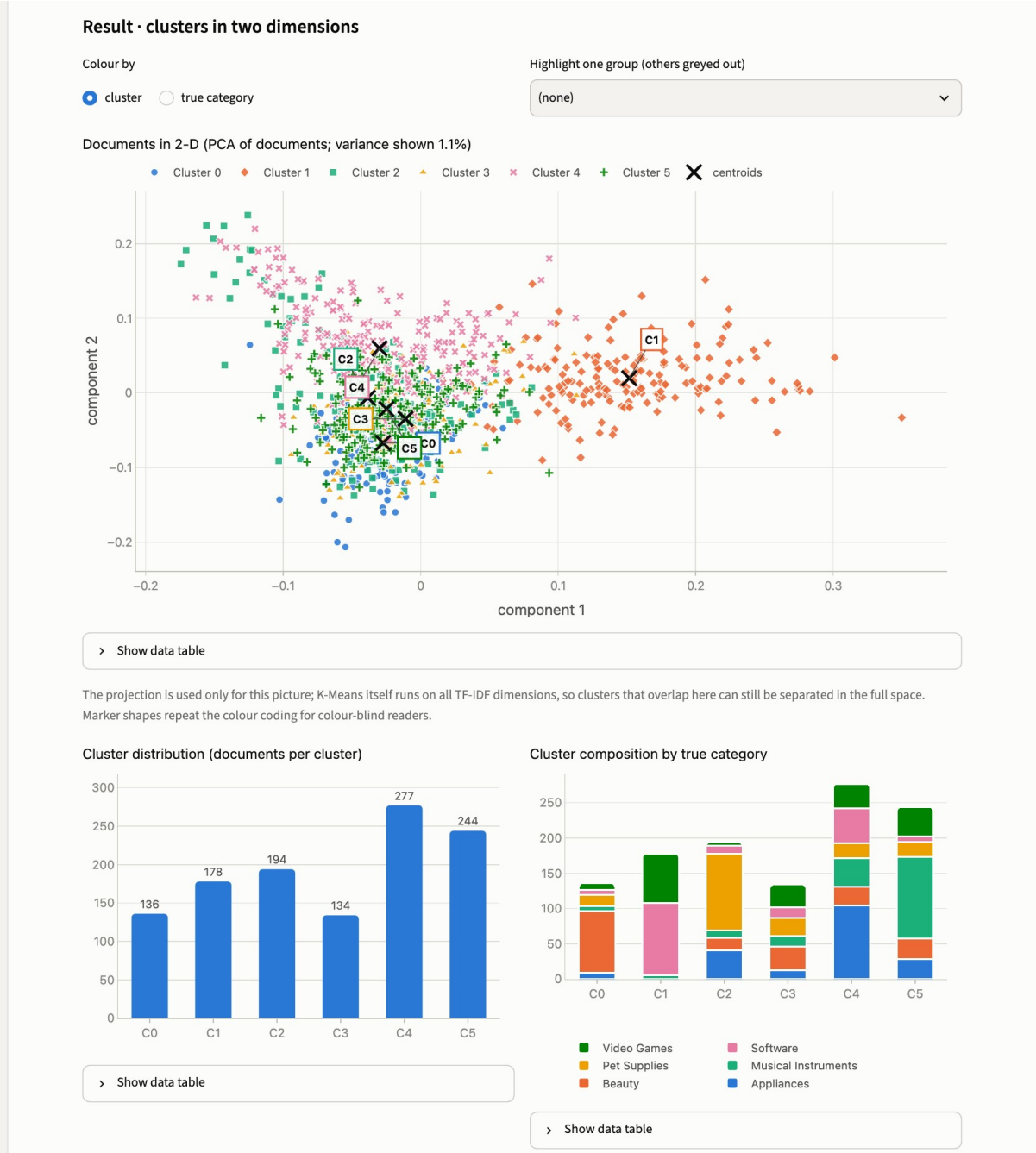
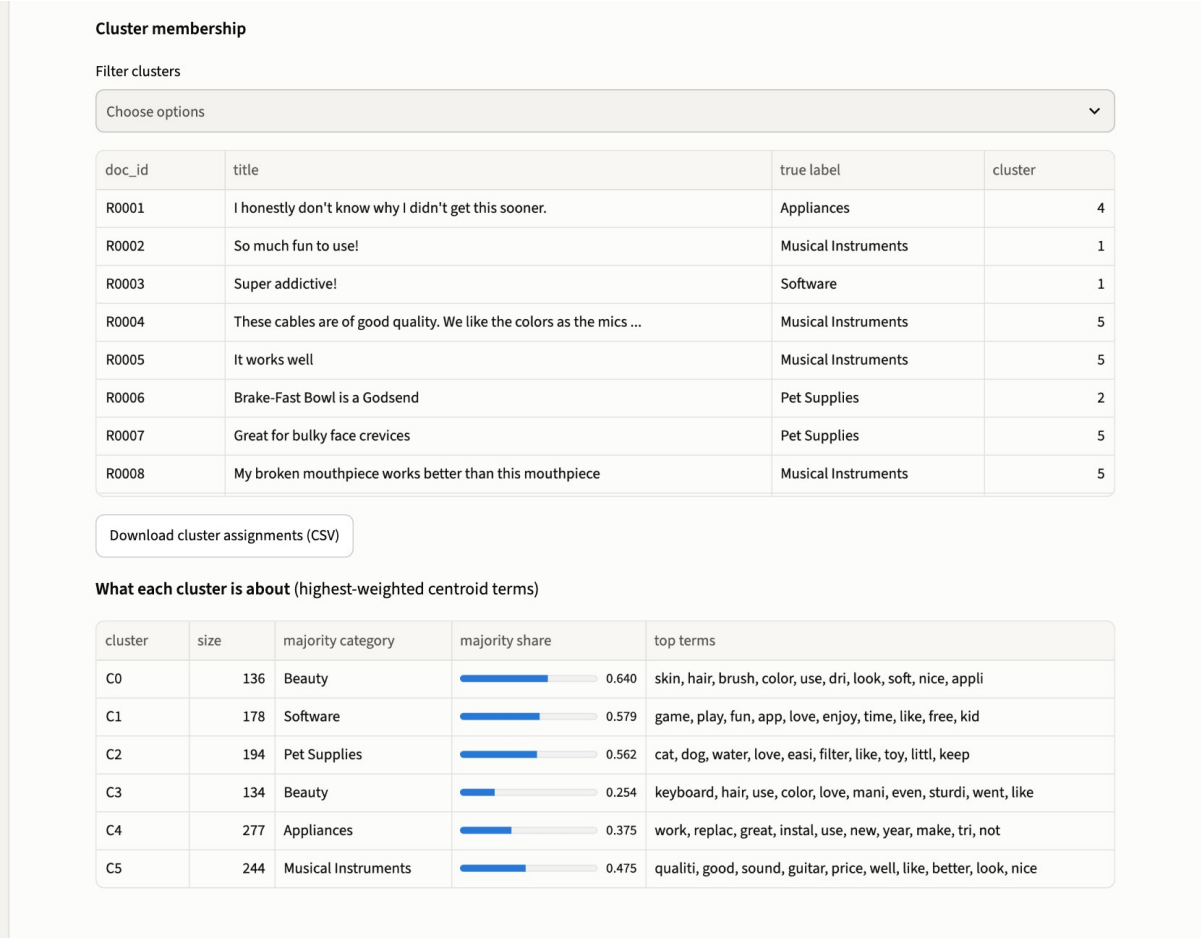


Figure 14. *2-D scatter plot of the documents coloured by K-Means cluster (PCA used for visualisation only; marker shapes repeat the colours), cluster distribution bar chart and cluster composition.*



Cluster membership

Filter clusters

Choose options

doc_id	title	true label	cluster
R0001	I honestly don't know why I didn't get this sooner.	Appliances	4
R0002	So much fun to use!	Musical Instruments	1
R0003	Super addictive!	Software	1
R0004	These cables are of good quality. We like the colors as the mics ...	Musical Instruments	5
R0005	It works well	Musical Instruments	5
R0006	Brake-Fast Bowl is a Godsend	Pet Supplies	2
R0007	Great for bulky face crevices	Pet Supplies	5
R0008	My broken mouthpiece works better than this mouthpiece	Musical Instruments	5

Download cluster assignments (CSV)

What each cluster is about (highest-weighted centroid terms)

cluster	size	majority category	majority share	top terms
C0	136	Beauty	0.640	skin, hair, brush, color, use, dri, look, soft, nice, appli
C1	178	Software	0.579	game, play, fun, app, love, enjoy, time, like, free, kid
C2	194	Pet Supplies	0.562	cat, dog, water, love, easi, filter, like, toy, littl, keep
C3	134	Beauty	0.254	keyboard, hair, use, color, love, mani, even, sturdi, went, like
C4	277	Appliances	0.375	work, replac, great, instal, use, new, year, make, tri, not
C5	244	Musical Instruments	0.475	qualiti, good, sound, guitar, price, well, like, better, look, nice

Figure 15. *Cluster membership table (document ID, title, true label, assigned cluster) and cluster descriptions.*

6.5 Robustness

Table 24. *K-Means variants: 10 single runs per variant (mean ± sd) and the best-RSS run of 10 restarts.*

Initialisation	Distance	Purity (single runs)	Rand Index (single runs)	Best-of-10 purity / RI
random	euclidean	0.430 ± 0.061	0.751 ± 0.022	0.539 / 0.789
random	cosine	0.431 ± 0.059	0.760 ± 0.018	0.488 / 0.781
k-means++	euclidean	0.417 ± 0.054	0.752 ± 0.016	0.475 / 0.766
k-means++	cosine	0.428 ± 0.052	0.760 ± 0.014	0.453 / 0.770

Table 25. *Sensitivity to the number of clusters k (mean of 3 seeds).*

k	2	3	4	5	6	7	8	9	10	11	12
RSS	1133.4	1129.9	1126.7	1124.0	1122.2	1120.0	1118.6	1116.7	1115.5	1114.0	1112.4
Purity	0.277	0.376	0.453	0.494	0.469	0.505	0.479	0.481	0.457	0.466	0.467
Rand Index	0.413	0.623	0.647	0.725	0.758	0.780	0.786	0.790	0.793	0.798	0.802

Single K-Means runs vary considerably (purity sd ≈ 0.061), and none of the four variants is consistently better: differences between them are smaller than the seed-to-seed variation, which is why ten restarts are used and why the report quotes mean ± sd. RSS falls steadily as k grows and the Rand Index rises (more "different-cluster" pairs), so neither can choose k on its own; k is fixed to the number of departments for the comparison with Naive Bayes.

7. Comparison of classification and clustering results

Table 26. *Naive Bayes (classification metrics) vs K-Means (Purity, Rand Index), default configuration.*

Method	Metric	Score	Chance level	Over 10 splits / seeds
Naive Bayes (supervised)	accuracy	0.774	0.167	0.782 ± 0.024
	macro precision / recall	0.774 / 0.774	0.167	
	macro F1	0.774	0.167	0.783 ± 0.024
K-Means (unsupervised)	purity	0.475	0.198	0.469 ± 0.044
	Rand Index	0.766	0.715	0.754 ± 0.016

Naive Bayes uses the 875 labelled training reviews and optimises exactly the target, so it can rely on department-specific words (dog, guitar, filter, xbox) and reaches 77.4% accuracy. K-Means never sees a label: it minimises RSS and therefore groups reviews by whatever dominates their vocabulary. Some clusters match a department well (C0 Beauty 0.640, C1 Software 0.579, C2 Pet Supplies 0.562), others collect cross-department themes: C1 ("game / play / fun / app") merges 103 Software and 70 Video Games reviews; C4 ("work / replac / great / instal") collects functional feedback from several departments (majority share only 0.375). Purity therefore stays at 0.475, far above chance (0.198) but well below the classifier.

The metrics are not interchangeable: accuracy and F1 compare a prediction with its label one by one, whereas purity and the Rand Index must cope with unnamed clusters. Purity equals the accuracy of a classifier that labels every cluster with its majority category, which makes it the closest counterpart to accuracy. It reaches 1 when every document forms its own cluster (k = N), so it is only comparable at a fixed k - in Table 25 it fluctuates between 0.457 and 0.505 for k = 5-12. The Rand Index is dominated by TN pairs. In practice the two methods complement each other: the classifier operationalises categories the team already has, while clustering exposes themes nobody defined in advance.

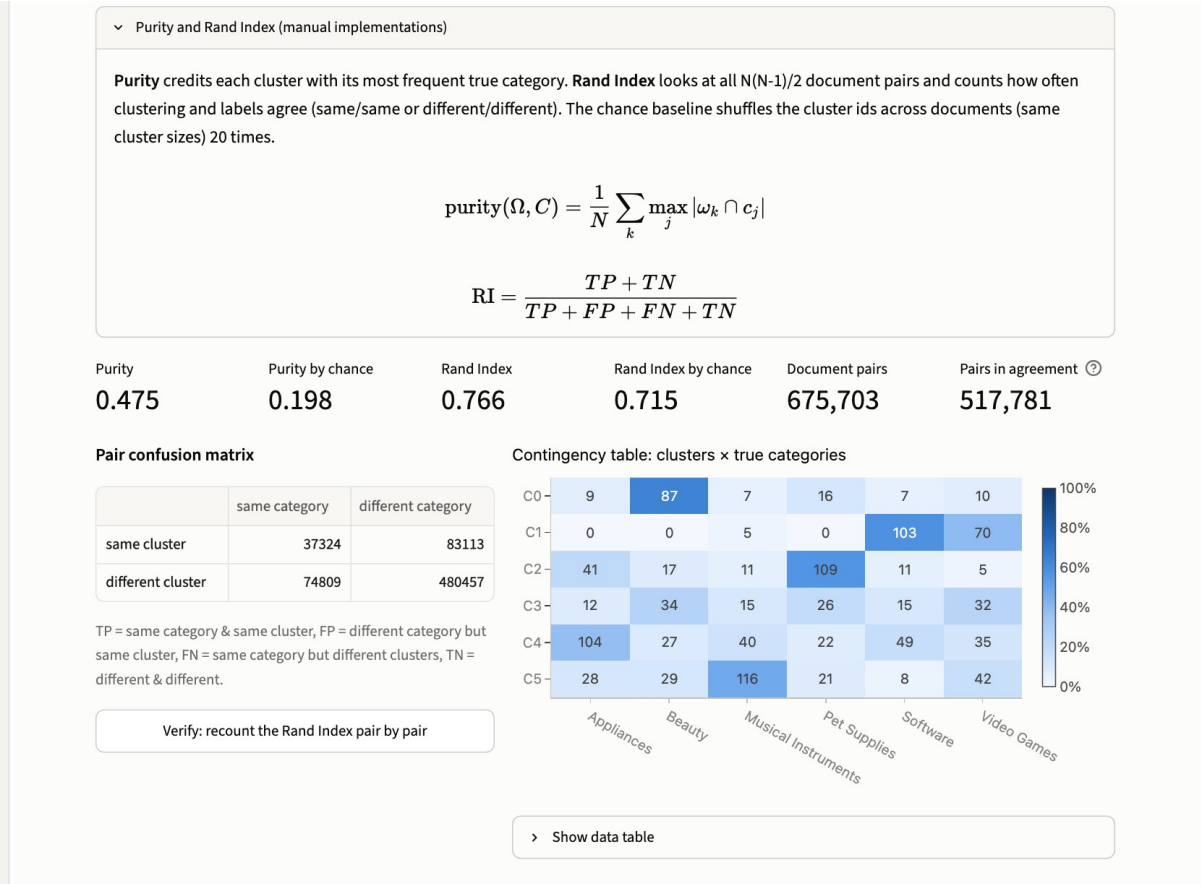
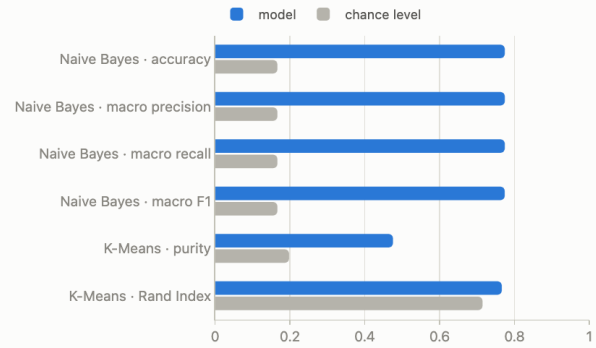


Figure 16. *Evaluation & comparison tab (1/2): Purity and Rand Index formulas, values and chance levels, pair confusion matrix and contingency heat map.*

Naive Bayes (supervised) vs K-Means (unsupervised)

Model scores vs chance level



method	metric	value	chance level
Naive Bayes	accuracy	0.774	0.167
Naive Bayes	macro precision	0.774	0.167
Naive Bayes	macro recall	0.774	0.167
Naive Bayes	macro F1	0.774	0.167
K-Means	purity	0.475	0.198
K-Means	Rand Index	0.766	0.715

Interpretation. Naive Bayes learns from 875 labelled reviews and routes **77.4%** of unseen reviews to the right department (4.6× the majority-class rate, macro-F1 0.774). K-Means never sees a label: its 6 clusters are dominated by one department for **47.5%** of the documents (purity; 2.4× the chance level of 19.8%). The Rand Index of **0.766** looks high, but a random clustering of the same sizes already scores **0.715**, because most of the 675,703 pairs are "different category, different cluster" - purity and the chance baseline are more informative here.

- **Supervised vs unsupervised** - Naive Bayes optimises exactly the target (the department), so it can use department-specific words (*dog, guitar, filter, game*). K-Means optimises compactness (RSS); it groups reviews by whatever dominates the vocabulary - its most mixed clusters are *"keyboard / hair / use"* (C3) and *"work / replac / great"* (C4), themes that cut across departments (the Software department also contains many app-store games).
- **Metrics are not interchangeable** - accuracy / F1 compare predictions with labels one-to-one; purity and Rand Index have to cope with clusters that have no names, so they reward any grouping that keeps categories together. Purity reaches 1 when every document is its own cluster (k = N), so it is only comparable at a fixed k; here k equals the number of categories.
- **Use together** - clusters reveal themes analysts did not define in advance; the classifier operationalises the categories they already care about.

Figure 17. *Evaluation & comparison tab (2/2): Naive Bayes vs K-Means against chance level, with the interpretation shown in the app.*

8. Recommendation results

8.1 Approach: content-based Top-K

The analyst selects a review (or types a new one). It goes through the same preprocessing as the corpus and becomes a unit-length TF-IDF vector with the corpus idf. Cosine scores are accumulated term-at-a-time through the inverted index and the K best documents are returned. Candidates whose cosine with the query, or with an earlier recommendation, is at least 0.80 are skipped (collapse near-duplicates), and optional filters restrict the list to the same category or a minimum score. For transparency each recommendation lists the shared terms with their contribution w(t,q)·w(t,d) to the score, together with rating, date and other metadata.

8.2 Top-K results

Query R0001 (Appliances) - "I honestly don't know why I didn't get this sooner.": I've had my Aeropress for 2+ years now and used primarily paper filters and once tried a sad and cheap mesh net filter, but this changes everything. This is an incredibly well manu…

Table 27. *Top-5 recommendations for R0001.*

Rank	doc_id	Score	Category	Title	Rating	Shared terms
1	R0945	0.215	Appliances	A Permanent Filter for a Small Coffeema…	5	mesh, filter, cup, well, use
2	R1192	0.167	Appliances	Easy to use	5	flavor, ground, recommend, not
3	R0385	0.161	Appliances	Since I do not like sediment left in my…	5	paper, filter, cup, use, not
4	R0754	0.158	Appliances	Mud on the bottom of the cup	3	filter, paper, better, well, not
5	R1217	0.158	Appliances	Water Tastes Awful	1	filter, manufactur, chang, tri, not

Free-text query: "The leash broke after a week and my dog got loose, the stitching is poor quality."

Table 28. *Top-5 recommendations for a typed query (not in the corpus).*

Rank	doc_id	Score	Category	Title	Rating	Shared terms
1	R0856	0.354	Pet Supplies	You want to regret it!	5	leash, dog
2	R0068	0.284	Pet Supplies	Broke after 2 uses	1	poor, broke, week, dog, qualiti
3	R1064	0.258	Pet Supplies	Good design	5	leash, dog
4	R1027	0.176	Pet Supplies	Excellent.	5	stitch, dog, qualiti
5	R0513	0.172	Pet Supplies	Prone to breaking but it does what it’s…	3	stitch, broke, qualiti

Query R0010 (Musical Instruments) - "Covers well and keeps dust off": a weak case.

Table 29. *Top-5 recommendations for R0010, where generic wording dominates.*

Rank	doc_id	Score	Category	Title	Rating	Shared terms
1	R1006	0.163	Beauty	Great scent and great price	5	pleasantli, surpris, not
2	R0795	0.134	Appliances	Unique and beautiful	5	pleasantli, surpris, cover, look, not
3	R0373	0.134	Musical Instruments	This bag has no padding. It is two laye…	2	fabric, cover, look
4	R0082	0.121	Musical Instruments	has a good weight so it lays out nicely…	4	fabric, lay, nice, qualiti, well
5	R0282	0.111	Appliances	Four Stars	1	fabric

The first two lists are useful: the coffee-maker review retrieves other reviews about coffee filters and sediment, and the typed leash complaint retrieves Pet Supplies reviews about leashes, breakage after a week and stitching quality - exactly the evidence an analyst investigating a defect needs. The third list shows the main weakness: the dust-cover review shares mostly generic phrases ("pleasantly surprised", "cover", "look"), so reviews from Beauty and Appliances outrank related Musical Instruments reviews.

8.3 Evaluation

Without user feedback, relevance is approximated by department agreement. Leave-one-out over all 1,163 reviews, **P@5 = 0.571** (share of Top-5 items from the query's department) and **MRR = 0.783** (mean reciprocal rank of the first same-department item); the top-ranked recommendation comes from the query's department for 67.8% of the queries (P@1). The chance level for P@1 and P@5 is about 0.167.

Table 30. *P@5 by department of the query.*

Department	Queries	P@5
Appliances	194	0.671
Beauty	194	0.587
Musical Instruments	194	0.506
Pet Supplies	194	0.535
Software	193	0.636
Video Games	194	0.493

8.4 Recommendation interface

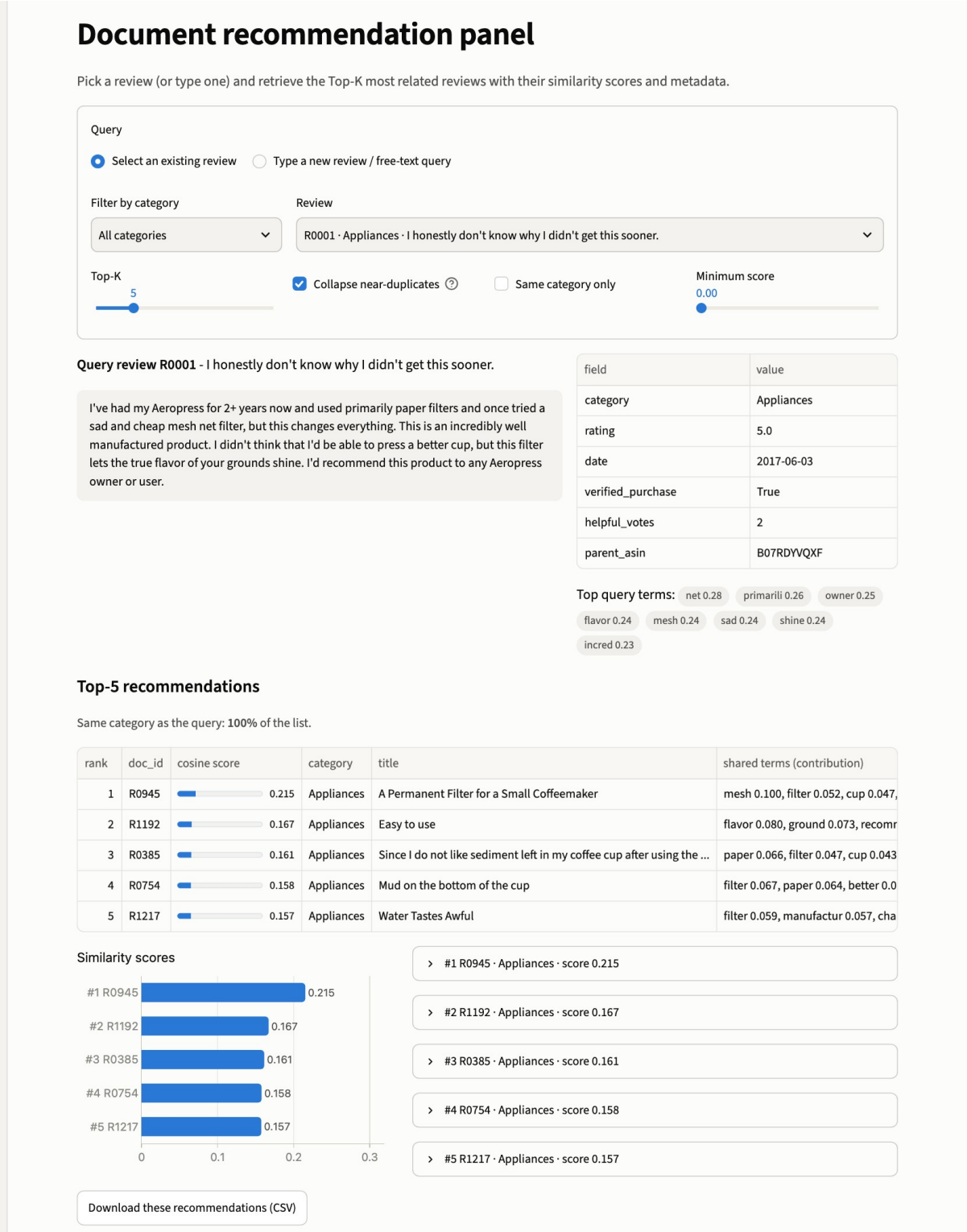


Figure 18. *Recommendation panel: query selection, options (Top-K, collapse near-duplicates, same category only, minimum score), the query with its metadata, and the Top-K list with cosine scores, metadata and shared terms.*

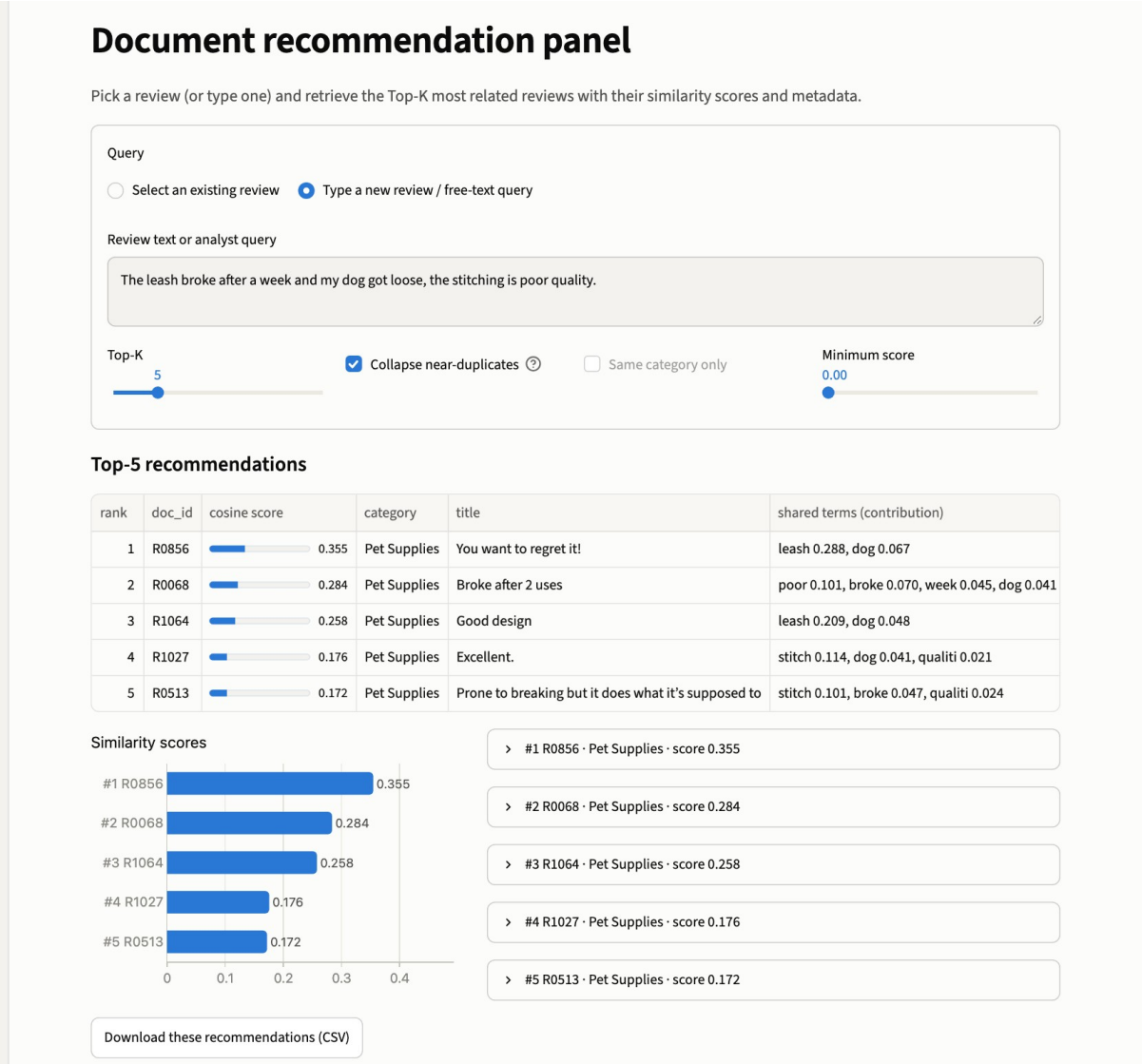


Figure 19. *Free-text mode: an analyst types a new complaint and receives related reviews.*

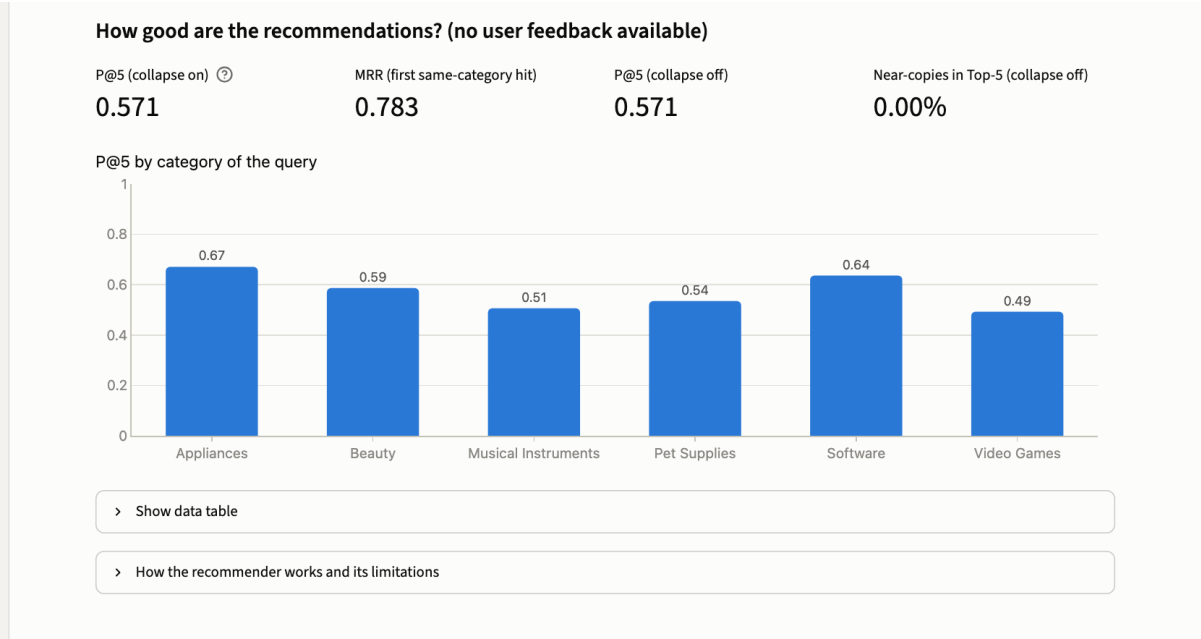


Figure 20. *Recommendation evaluation in the app: P@K with and without duplicate collapsing, MRR, and P@K by category.*

8.5 Findings and limitations

- Specific vocabulary (filter, leash, xbox, guitar) gives precise neighbours; Appliances and Software queries reach the highest P@5.
- Vocabulary mismatch: synonyms (leash/lead, broke/snapped) do not match, because TF-IDF has no notion of meaning.
- Short, generic reviews ("works great, love it") share only sentiment words and receive vague neighbours.
- Cross-department themes (games in Software and Video Games) lower department agreement, although such neighbours can be useful when investigating an issue.
- No personalisation or quality signal: helpful votes, recency and ratings are displayed but not used for ranking.
- P@K uses department agreement as a proxy for relevance; real relevance would need analyst judgements.

9. Inference and discussion (compulsory questions)

Q1. How did the choice of preprocessing steps affect the vocabulary, classification, clustering and recommendations?

Seven pipelines were evaluated on the same de-duplicated documents (Table 31). Naive Bayes is averaged over 10 random splits and K-Means over 10 seeds, because a single run of either can be lucky.

Table 31. *Preprocessing ablation (mean over 10 splits / seeds; sd of K-Means purity shown).*

Pipeline	Vocab (all / pruned)	Terms per doc	NB acc.	NB macro-F1	K-Means purity (sd)	K-Means RI	Rec P@5
A0 minimal: regex tokens only	8,532 / 3,878	74.7	0.717	0.719	0.399 (0.046)	0.729	0.519
A1 + markup removal & lower-casing	7,158 / 3,467	68.2	0.750	0.752	0.429 (0.037)	0.740	0.543
A2 + stop-word removal	7,021 / 3,334	34.0	0.776	0.776	0.407 (0.034)	0.742	0.548
A3 + Porter stemming (default)	4,987 / 2,591	34.0	0.782	0.783	0.469 (0.044)	0.754	0.571
A4 + lemmatisation instead of stemming	5,321 / 2,651	34.0	0.779	0.780	0.478 (0.057)	0.758	0.574
A5 + stemming + bigrams	37,518 / 5,000	67.0	0.759	0.761	0.408 (0.049)	0.746	0.515
A6 + stemming + 1-3 grams	74,382 / 5,000	99.0	0.756	0.758	0.398 (0.050)	0.744	0.509

- **Vocabulary.** Cleaning and regex tokenisation cut the raw vocabulary from 13,000 to 8,328 types; lower-casing to 7,158; stop-word removal hardly changes the number of types but halves the tokens (79,295 → 39,556, about 34 terms per review); Porter stemming merges inflections (7,021 → 4,987). Bigrams and trigrams multiply the vocabulary to 37,518 and 74,382 terms.
- **Classification.** Naive Bayes accuracy rises from 0.717 (minimal) to 0.750 with markup removal and lower-casing, 0.776 with stop-word removal and 0.782 with stemming. Lemmatisation is equivalent (0.779); n-grams lower it to 0.759 / 0.756: 91.4% of the 25,065 word pairs in the 875 training reviews occur only once (unigrams: 44.3%), so they add sparse features that rarely help on unseen reviews.
- **Clustering.** Purity moves between 0.398 and 0.478, while its seed-to-seed sd is 0.03-0.06, so most differences are within noise. The clearest gain comes from stemming / lemmatisation (0.407 → 0.469 / 0.478), which lets reviews that use different inflections share dimensions. Stop-word removal did not improve clustering on average (0.429 → 0.407), and n-grams made it worse (0.408 / 0.398).

- **Recommendation.** P@5 improves step by step from 0.519 to 0.571 (lemmatisation 0.574) and MRR from 0.730 to 0.783: merging surface variants increases the overlap between related reviews, whereas n-grams (0.515 / 0.509) make vectors sparser so fewer neighbours share any feature. Without markup removal the meaningless token "br" is shared by 224 documents and creates false similarity.

Conclusion: for short, noisy reviews the best pipeline is markup removal + lower-casing + stop words (keeping negations) + stemming on unigrams. It gives the best classification and near-best recommendation with the smallest vocabulary. Keeping negations makes "not" the most frequent term; they were kept because they carry sentiment when analysts read term lists, and their effect on the scores was not measured separately.

Q2. How can duplicate or near-duplicate documents affect the results, and how were they detected and handled?

Detection. Exact copies are found by hashing the normalised text; near-duplicates by TF-IDF cosine ≥ 0.80, merged into groups with union-find (Section 4.5). Against the 62 ground-truth pairs the detector reached precision 1.000 and recall 1.000; the gap between the least similar true pair (0.886) and the most similar unrelated pair (0.496) makes the threshold robust between 0.5 and 0.85 (Table 12).

Effects measured (Table 32; three settings, Naive Bayes over 10 random splits):

Table 32. *Duplicate study.*

Setting	Docs	NB accuracy	Test docs with copy in training	NB acc. on those / others	K-Means purity / RI	P@5 no collapse (copies in Top-5)	P@5 collapse
S1 keep duplicates, random split	1224	0.793 ± 0.023	246	96.7% / 77.8%	0.469 / 0.749	0.583 (2.0%)	0.570
S2 keep duplicates, duplicate-aware split	1224	0.780 ± 0.029	0	-	0.469 / 0.749	0.583 (2.0%)	0.570
S3 remove near-duplicates (default)	1163	0.782 ± 0.024	0	-	0.469 / 0.754	0.571 (0.0%)	0.571

- **Classification (leakage).** With duplicates kept and a plain random split, 246 test reviews (over 10 splits) had a copy in the training set. Naive Bayes classified **96.7%** of them correctly versus **77.8%** for the other test reviews - it partly remembers the copy - and the overall accuracy is inflated to 0.793 compared with 0.780 for a duplicate-aware split and 0.782 after de-duplication.
- **Clustering.** Exact copies always land in the same cluster and add "free" same-cluster pairs, and in template-heavy collections they can form micro-clusters that attract centroids. In this corpus duplicates are only 9.9% of the documents, and the measured effect is negligible (purity 0.469 in all settings; Rand Index 0.749 vs 0.754).
- **Recommendation.** Without collapsing, 2.0% of all Top-5 slots are copies of the query. They inflate P@5 (0.583 vs 0.570) because a copy always has the same department, yet they give the analyst no new information.

Handling. By default one document per duplicate group is kept before any analysis (61 documents removed), the train/test split is duplicate-aware, and the recommender collapses near-duplicates of the query and of earlier recommendations. All three policies can be switched in the app to reproduce the comparison.

Q3. Which recommendation approach was selected, and how do content-based and collaborative recommendation compare?

ReviewLens uses **content-based recommendation**: related reviews are those whose TF-IDF vectors have the highest cosine similarity with the query review (Section 8). Table 33 compares it with collaborative filtering.

Table 33. *Content-based vs collaborative recommendation.*

Aspect	Content-based (implemented)	Collaborative filtering
Data required	the item content (review text) and a representation (TF-IDF)	a user × item interaction matrix (ratings, clicks, purchases, reads)
How relatedness is found	similar content: shared, weighted terms	similar behaviour: items liked by the same users (item-item) or users with similar histories (user-user)
New item (cold start)	works immediately - only its text is needed	cannot recommend an item until users have interacted with it
New / anonymous user	no history needed (query = a document or text)	needs the user's history
Strengths	transparent (shared terms), domain-specific, no user data / privacy issues	captures relations not visible in the text; serendipitous, popularity-aware
Weaknesses	vocabulary mismatch, over-specialisation (more of the same)	sparsity, popularity bias, cold start, needs many active users
Preferable when	rich text, few or no interactions, new content daily (reviews, news, papers, tickets)	dense interaction logs, stable catalogue, goal is personalisation (e-commerce, streaming)

Evidence for this dataset. The reviews come with anonymised user and product IDs, but the 1,224 reviews are written by 1,105 different users; only 62 users wrote more than one review, only 64 of 1,059 products were reviewed by more than one user, and the user × item matrix is 0.10% dense. This sparsity is partly a result of the sampling (about 200 reviews per department, at most two per user); the full Amazon dataset has far more interactions, so the conclusion applies to this corpus. Here collaborative filtering would have almost no co-occurrence signal, whereas every review has text. Moreover the users of ReviewLens are analysts, not shoppers: they want reviews about the same issue, which is a content question. A hybrid would become attractive once analyst feedback is logged (e.g. "this related review was useful"), which could re-rank content-based candidates with item-item collaborative signals.

Q4. How do text mining, classification, clustering and recommendation together help domain experts decide better?

- **Routing (classification).** New reviews are assigned to a department automatically with 77.4% accuracy (Appliances F1 0.878); the posterior confidence lets the team auto-route confident cases and send uncertain ones (often Software vs Video Games) to manual triage.
- **Theme discovery (clustering).** K-Means groups reviews into themes without predefined labels, and the share of 1-2 star reviews per cluster ranks them by dissatisfaction: cluster C4 (work, replac, great, instal, use, new, year, make) has the highest share of negative reviews (31.0%, mean rating 3.46), cluster C1 the lowest (16.9%). Such cross-department themes (installation, replacement parts) are invisible in a department-by-department view.
- **Root-cause vocabulary (text mining).** For every department the log-odds of terms in 1-2 star vs 4-5 star reviews shows what unhappy customers talk about (Table 34; indicative only, because each department has just 36-48 negative reviews), for example Appliances: sensor, silverwar, return, advertis; Beauty: shave, shaver, bristl, cartridg; Musical Instruments: broken, hit, poor, soon; Pet Supplies: crumb, poorli, gourmet, healthi; Software: credit, norton, answer, follow; Video Games: ban, crap, found, offic.

- **Investigation (recommendation).** Given one complaint (e.g. a broken leash), the analyst immediately sees similar reviews, their ratings and dates, which helps to judge whether it is an isolated case or a recurring defect, and to collect evidence for the supplier.
- **Data quality.** Duplicate detection removes re-posts that would otherwise overstate a problem's frequency and inflate model metrics.

Table 34. *Department vocabulary and what negative (1-2 star) reviews talk about (stemmed terms).*

Department	Distinctive terms (vs other departments)	Share 1-2 star	Negative reviews talk about
Appliances	coffe, refriger, stove, burner, drain, keurig, ice, ge	23.2%	sensor, silverwar, return, advertis, compani, disappoint, fact, fork
Beauty	curl, moistur, lotion, shave, blond, hydrat, shaver, lash	24.7%	shave, shaver, bristl, cartridg, final, norelco, tape, defect
Musical Instruments	amp, instrument, guitar, karaok, record, tuner, reverb, violin	19.6%	broken, hit, poor, soon, 5mm, anchor, clamp, event
Pet Supplies	puppi, litter, dog, collar, lb, cage, feed, fur	21.6%	crumb, poorli, gourmet, healthi, tell, bare, clip, compani
Software	kindl, tax, glitch, hdd, norton, corel, quicken, data	18.7%	credit, norton, answer, follow, form, last, turbotax, uninstal
Video Games	consol, xbox, wii, gamer, nintendo, razer, scroll, madden	20.6%	ban, crap, found, offic, american, garbag, naga, steam

Table 35. *K-Means themes ranked by share of 1-2 star reviews.*

Cluster	Docs	Mean rating	Share 1-2 star	Majority category	Top terms
C4	277	3.46	31.0%	Appliances	work, replac, great, instal, use, new, year, make
C5	244	3.91	21.3%	Musical Instruments	qualiti, good, sound, guitar, price, well, like, better
C0	136	3.93	18.4%	Beauty	skin, hair, brush, color, use, dri, look, soft
C3	134	4.03	17.2%	Beauty	keyboard, hair, use, color, love, mani, even, sturdi
C2	194	4.09	17.0%	Pet Supplies	cat, dog, water, love, easi, filter, like, toy
C1	178	3.89	16.9%	Software	game, play, fun, app, love, enjoy, time, like

Q5. Naive Bayes vs K-Means: findings, limitations and improvements for real-world deployment

Comparison. With the appropriate metrics (Table 26), the supervised classifier achieves accuracy 0.774 and macro-F1 0.774 (0.782 ± 0.024 over 10 splits), 4.6× the majority-class rate. The unsupervised K-Means reaches purity 0.475 (0.469 ± 0.044 over seeds) and Rand Index 0.766, compared with 0.198 and 0.715 for random clusterings. Labels are clearly recoverable from text, but the dominant structure of the vocabulary is only partly aligned with departments.

Key findings

- Markup removal, stop-word removal and stemming improve classification and recommendation; n-grams hurt on 30-350-word reviews.
- Duplicates inflate accuracy through train/test leakage (96.7% vs 77.8% on affected vs other test reviews); de-duplication or a duplicate-aware split is necessary for honest evaluation.
- Software and Video Games overlap heavily (app-store games), which is the largest single confusion (15 of 65 errors) and produces the merged game cluster.
- K-Means is sensitive to initialisation (purity sd ≈ 0.05); restarts and reporting mean ± sd are essential. The Rand Index is inflated by TN pairs and must be read against its chance level.

- Content-based recommendation retrieves same-department reviews 57.1% of the time (MRR 0.783) and is the practical choice for this sample, whose interactions are too sparse for collaborative filtering.

Limitations

- Moderate corpus size (1,224 reviews, about 145 training reviews per class) and short texts limit both models; results are for six departments only.
- Bag-of-words features ignore word order and meaning (synonyms, negation scope); multinomial NB assumes conditional independence.
- Department labels are a proxy for relevance in P@K and the target for clustering evaluation, although clusters may legitimately follow other themes.
- 24 of the duplicates were injected to obtain ground truth; real duplicate rates and types vary by platform.
- The 2-D projection keeps only about 1% of the variance, so the scatter plot understates the separation in the full space.

Suggested improvements for deployment

- Train on far more labelled reviews and add Complement NB or a linear model; calibrate confidences to decide auto-routing vs manual triage.
- Use sentence embeddings or LSA for semantic similarity (synonyms) in clustering and recommendation; keep TF-IDF for explainability.
- Spherical K-Means or hierarchical clustering with k chosen by silhouette / stability; track themes over time to detect emerging issues.
- Scale duplicate detection with MinHash / LSH, and apply it before every retraining.
- Log analyst feedback to build a hybrid recommender; add recency and helpful-vote boosts to ranking.
- Monitor drift (new products, vocabulary), retrain periodically, and protect privacy (user IDs are already anonymised).

10. Limitations and suggested improvements

The main limitations and improvements are discussed in Q5 (Section 9) and in Section 8.5 for the recommender. In short: the corpus is moderate in size and covers six departments; bag-of-words features do not capture meaning; K-Means is unstable across initialisations; and P@K uses department agreement as relevance. Semantic representations, larger labelled data, more robust clustering, scalable duplicate detection and analyst feedback for a hybrid recommender are the recommended next steps.

11. Evidence of execution (local machine)

The brief names the BITS Virtual Lab as the execution environment. Execution on a local machine was allowed for this submission, so the evidence below was captured on a group member's Mac.

The evidence comes from the submitted package itself: G47_Assignment2.zip was unzipped into an empty folder, a new virtual environment was created, requirements.txt was installed, the tests and all experiments were re-run, and the app was started with streamlit run. The complete log is evidence/execution_log.txt.

Table 36. *Environment of the execution run.*

Item	Version	Item	Version
Operating system	macOS 27.0.1 arm64	Python	3.14.6
streamlit	1.67.0	pandas	3.1.1
numpy	2.5.4	plotly	7.1.0
nltk	3.10.4	scikit-learn	not installed

Excerpt of the log, copied verbatim (long outputs are shortened where marked):

```
$ date
Fri Nov  6 18:18:41 IST 2026
$ sw_vers
ProductName:            macOS
ProductVersion:         27.0.1
BuildVersion:           26A434
$ uname -m
arm64
$ python3 --version
Python 3.14.6
$ unzip -q ../G47_Assignment2.zip
$ pwd
/Users/fiftyfivetech/shekhar-personal/ir_ass/clean_run/G47_Assignment2
$ ls
app.py
data
G47_Contribution.xlsx
README.md
report
requirements.txt
results
scripts
tests
textmining
views
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install --disable-pip-version-check -r requirements.txt
[... 122 lines shortened here; complete text in evidence/execution_log.txt ...]
Successfully installed MarkupSafe-3.0.4 altair-6.3.0 anyio-4.16.0 attrs-26.1.0 certifi-2026.10.23
charset_normalizer-3.5.2 click-8.5.0 cloudpickle-3.1.2 defusedxml-0.7.1 h11-0.16.0 idna-3.20
itsdangerous-2.2.0 jinja2-3.1.6 joblib-1.6.0 jsonschema-4.26.0 jsonschema-specifications-2025.9.1
narwhals-2.28.0 nltk-3.10.4 numpy-2.5.4 packaging-26.3 pandas-3.1.1 pillow-13.0.0 plotly-7.1.0
protobuf-7.36.2 pyarrow-26.0.1 pydeck-0.9.3 python-dateutil-4.0.0 python-multipart-0.0.33
referencing-0.37.0 regex-2026.9.29 requests-2.34.2 rpds-py-2026.9.1 starlette-1.10.0
streamlit-1.67.0 tqdm-4.70.1 typing-extensions-4.16.0 tzdata-2026.6 urllib3-2.8.0 uvicorn-0.54.0
websockets-17.4
$ pip freeze | grep -iE '^(streamlit|pandas|numpy|plotly|nltk)=='
nltk==3.10.4
numpy==2.5.4
pandas==3.1.1
plotly==7.1.0
streamlit==1.67.0
$ pip freeze | grep -ic scikit-learn
0
$ python tests/test_core.py
PASS  test_duplicate_detection
PASS  test_index_scoring_equals_matrix_cosine
PASS  test_inverted_index_and_tfidf_by_hand
PASS  test_kmeans_separates_two_blobs
PASS  test_metrics_by_hand
PASS  test_naive_bayes_textbook_example
PASS  test_preprocessing_pipeline
PASS  test_purity_textbook_example
PASS  test_rand_index_textbook_example
PASS  test_rand_index_two_methods_agree
PASS  test_stratified_split_keeps_groups_together
PASS  test_tfidf_textbook_example

12/12 tests passed
$ python scripts/run_experiments.py
[... 88 lines shortened here; complete text in evidence/execution_log.txt ...]
done in 114.6s
$ python -c "import json;s=json.load(open('results/summary.json'));print(round(s['naive_bayes']
['accuracy'],4),round(s['kmeans']['purity'],4),round(s['kmeans']
['rand_index'],4),round(s['recommendation']['precision_at_k'],4))"
0.7743 0.4755 0.7663 0.5713
$ streamlit run app.py
2026-11-06 18:20:53.155 Uvicorn server started on :::8501
  You can now view your Streamlit app in your browser.
  Local URL: http://localhost:8501
  For better performance, install the Watchdog module:
```

```
  $ xcode-select --install
  $ pip install watchdog

$ curl -s http://localhost:8501/_stcore/health
ok
$ date
Fri Nov  6 18:20:54 IST 2026
```

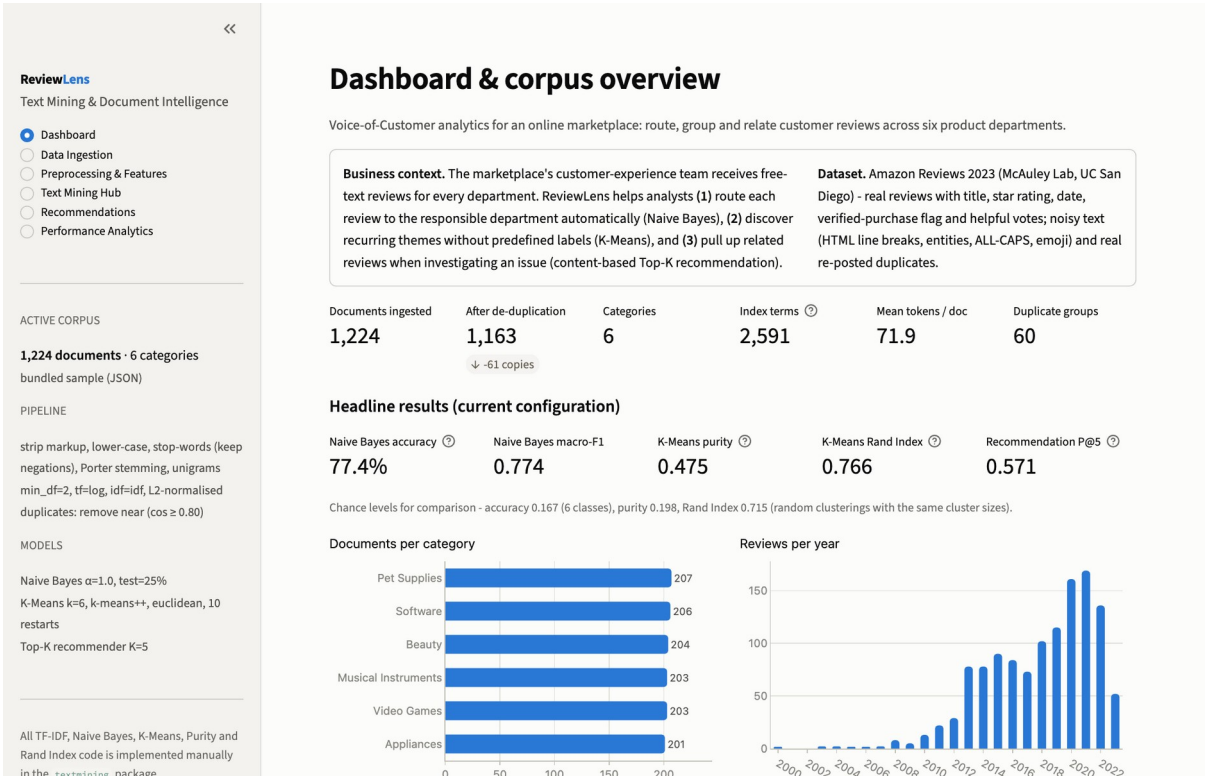


Figure 21. *Local run at http://localhost:8501 - dashboard page.*

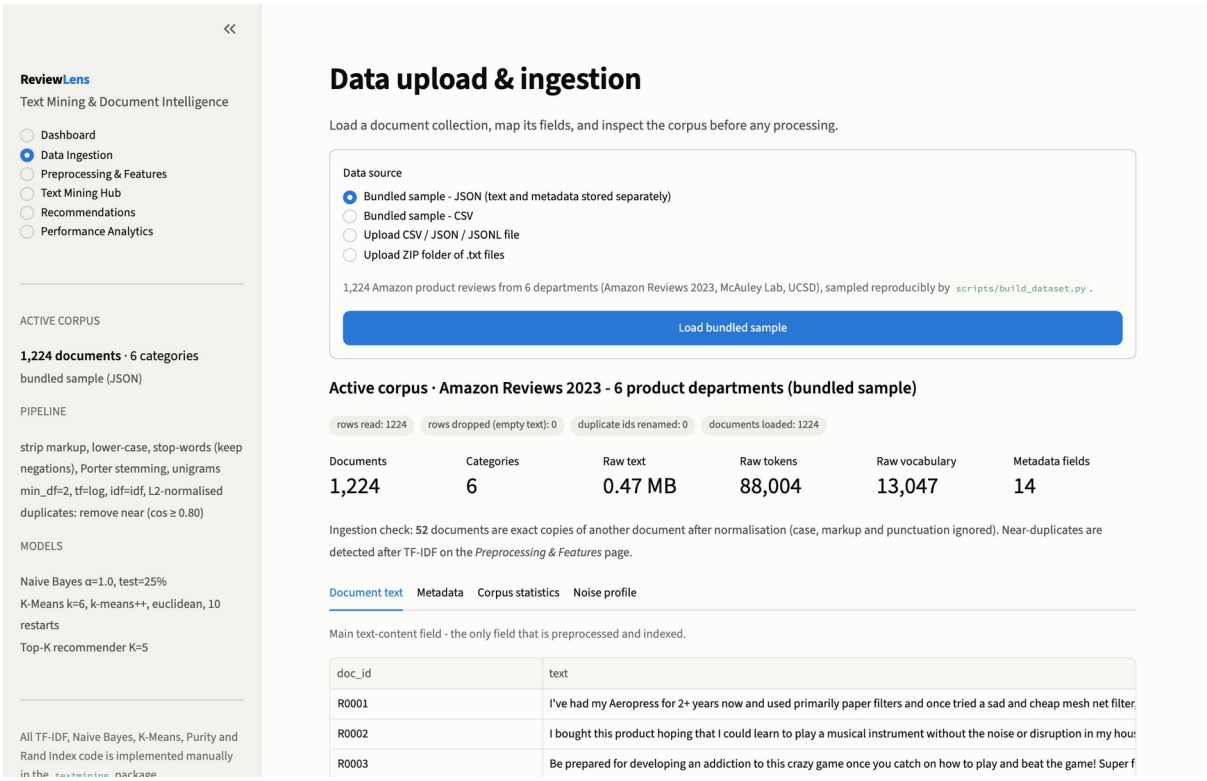


Figure 22. *Local run at http://localhost:8501 - data ingestion page.*

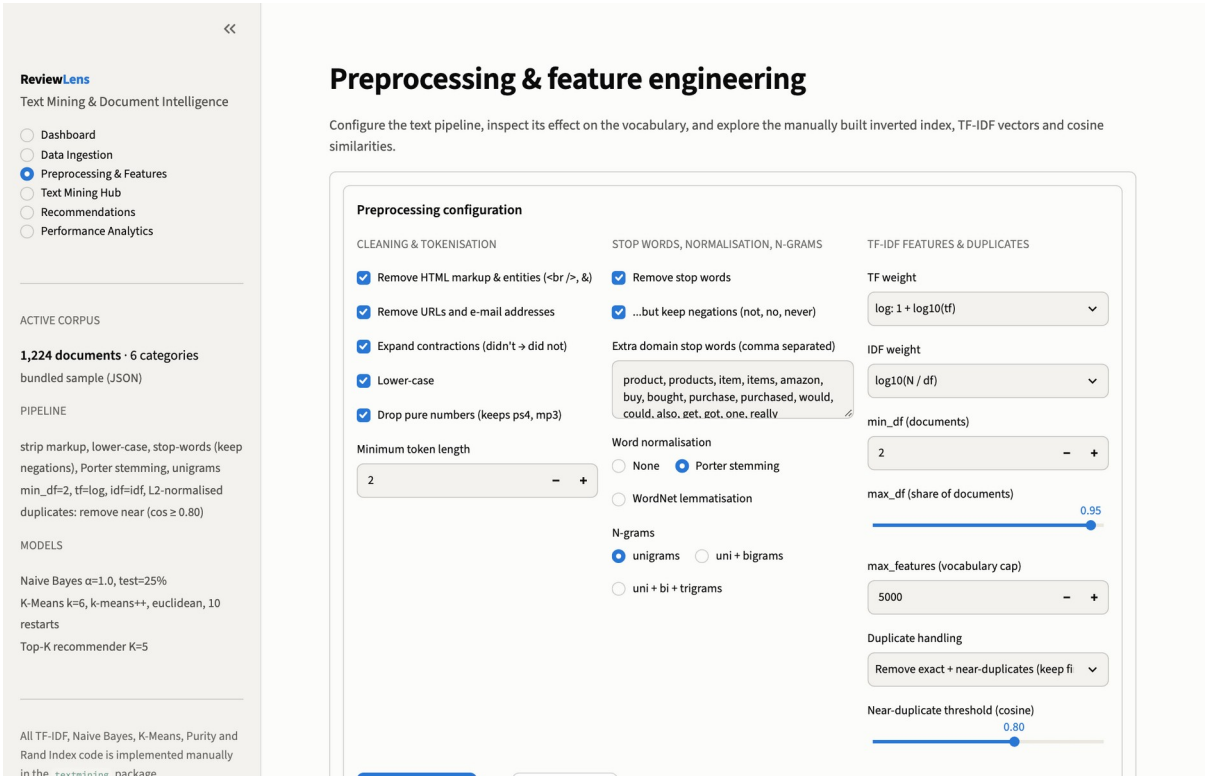


Figure 23. *Local run at http://localhost:8501 - preprocessing page.*

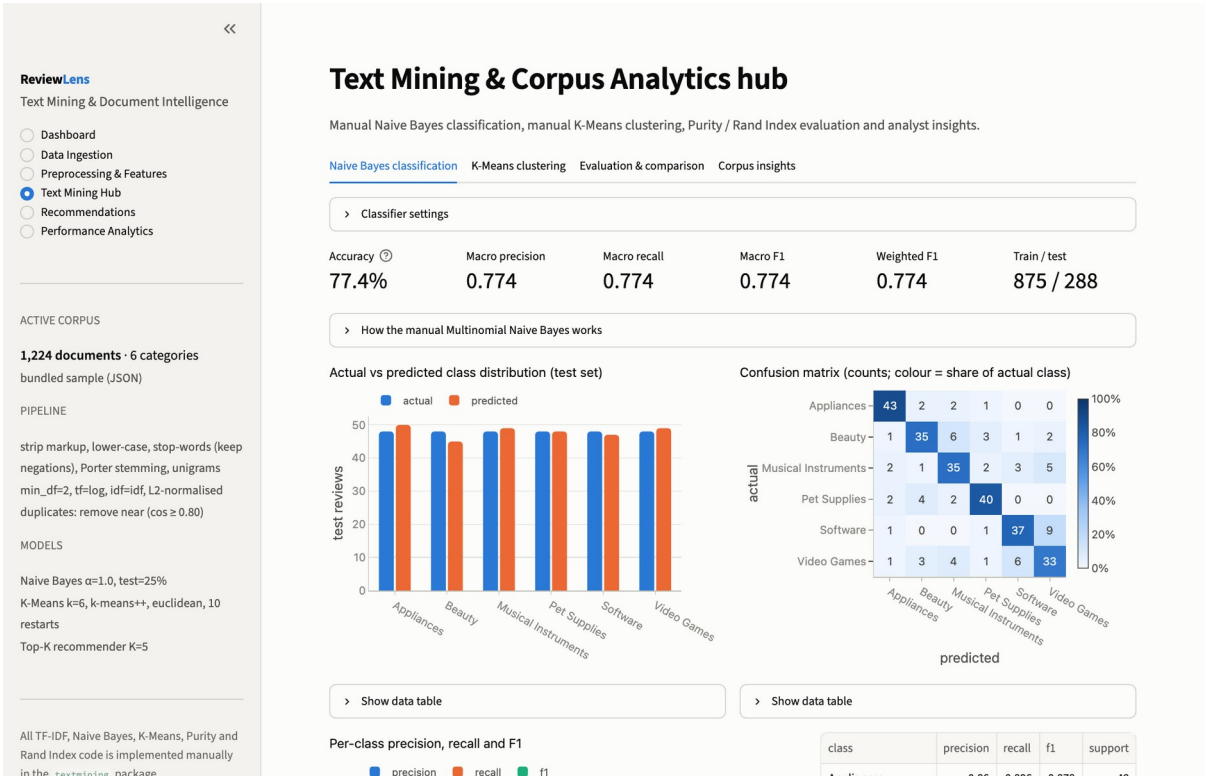


Figure 24. *Local run at http://localhost:8501 - text mining hub page.*

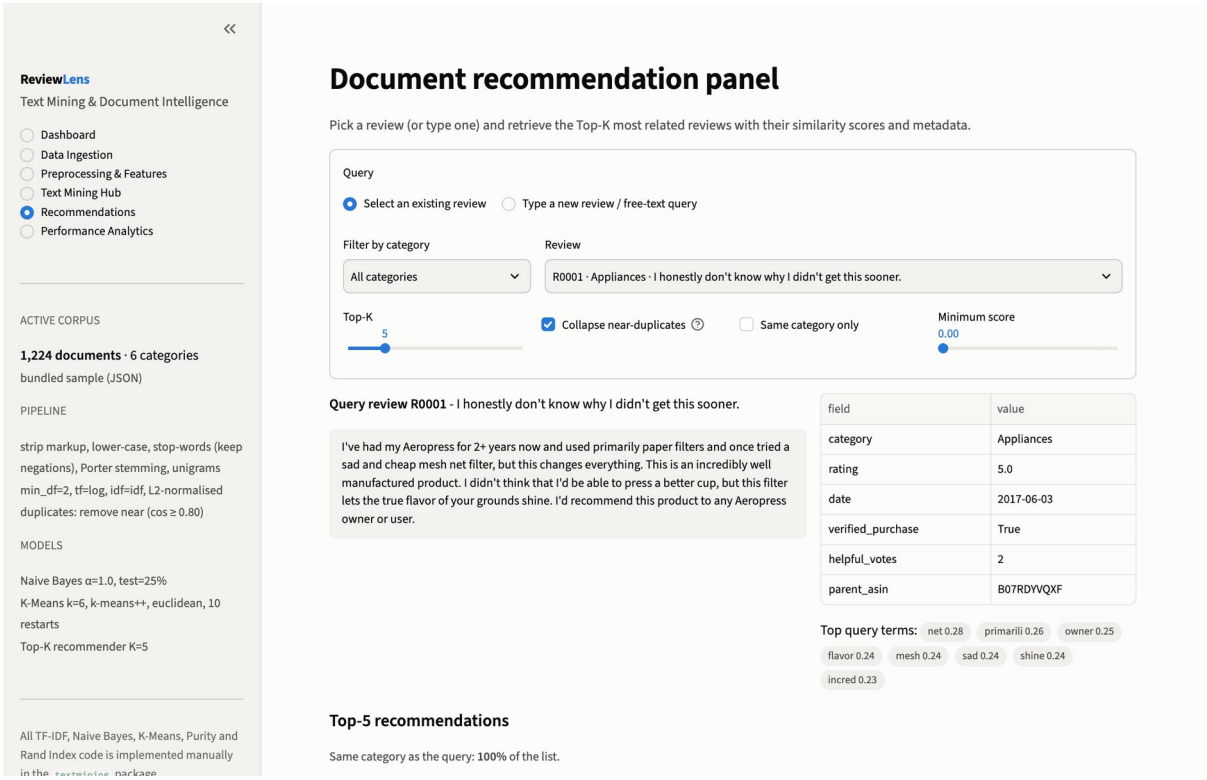


Figure 25. *Local run at http://localhost:8501 - recommendations page.*

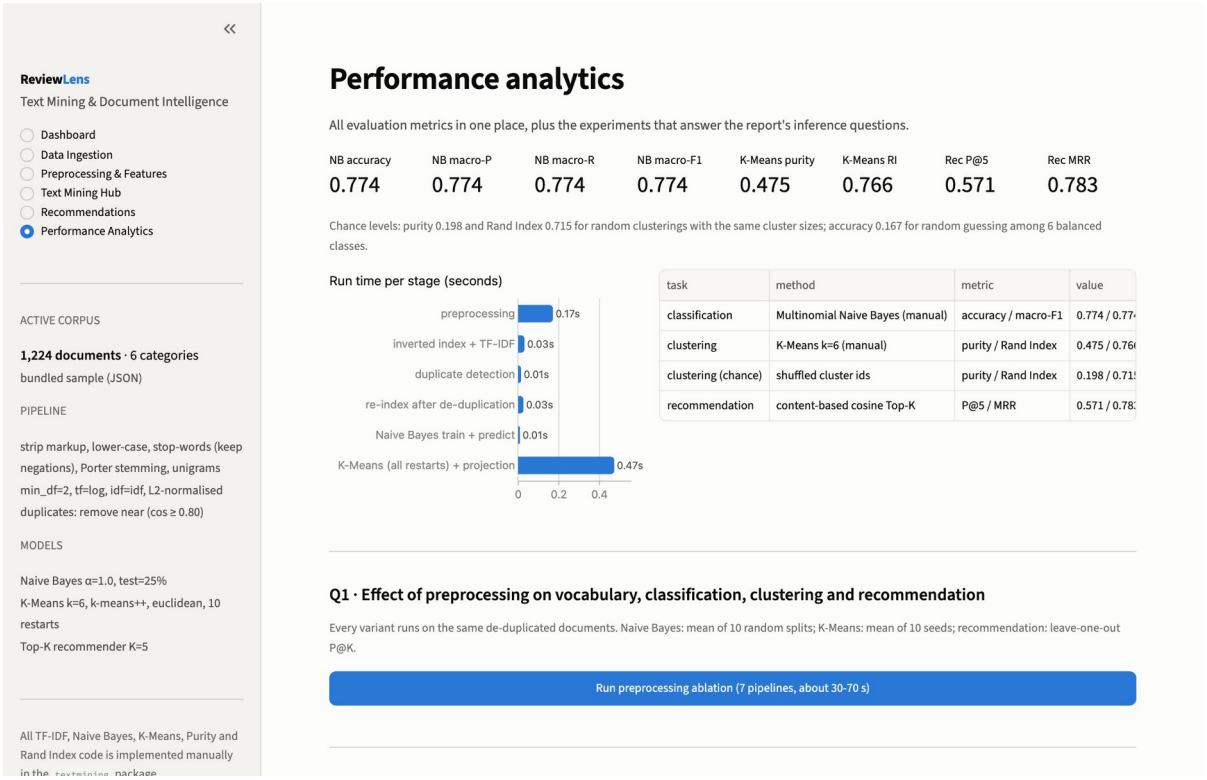


Figure 26. *Local run at http://localhost:8501 - performance analytics page.*

Screen recording: evidence/G47_demo.mp4 (4:55 min, 22.2 MB, no audio). It is an automated, unnarrated browser walkthrough of the app running at http://localhost:8501 on the same machine, driven by a script and captioned step by step. It starts with the execution log and then shows every page, including a file upload, a change of the preprocessing configuration, classification, clustering, recommendation and the duplicate study.

All application screenshots in this report were taken from this run.

12. Running the application and submission contents

Installation and start (see README.md for details):

```
python3 -m venv .venv  &&  source .venv/bin/activate
pip install -r requirements.txt       (streamlit, pandas, numpy, plotly, nltk)
streamlit run app.py                  → http://localhost:8501
python tests/test_core.py             (12 correctness tests)
python scripts/run_experiments.py     (regenerates every number of this report in results/)
```

Table 37. *Contents of G47_Assignment2.zip.*

Item	Location
Streamlit application source (.py)	app.py, views/*.py, textmining/*.py, .streamlit/config.toml
Supporting files	scripts/ (dataset builder, experiment runner), tests/, results/ (CSV/JSON outputs)
Dataset	data/amazon_reviews_corpus.json (text + separate metadata), data/amazon_reviews_corpus.csv, data/DATASET_CARD.md
Report (PDF)	report/G47_Report.pdf (this document)
Execution evidence	evidence/ (execution log, screenshots and screen recording)
README	README.md (installation and run commands)
Contribution sheet	G47_Contribution.xlsx

References

Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press. (Ch. 6 TF-IDF and the vector space model; Ch. 13 Naive Bayes; Ch. 16 flat clustering, purity and Rand Index; Ch. 19.6 near-duplicates.)

Hou, Y., Li, J., He, Z., Yan, A., Chen, X., & McAuley, J. (2024). Bridging Language and Items for Retrieval and Recommendation. arXiv:2403.03952. Dataset: https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023

Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. Proc. ACM-SIAM SODA, 1027-1035.

Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130-137. (NLTK implementation used.)

Streamlit (https://streamlit.io), Plotly (https://plotly.com/python), NLTK (https://www.nltk.org).

Appendix A. Additional screenshots

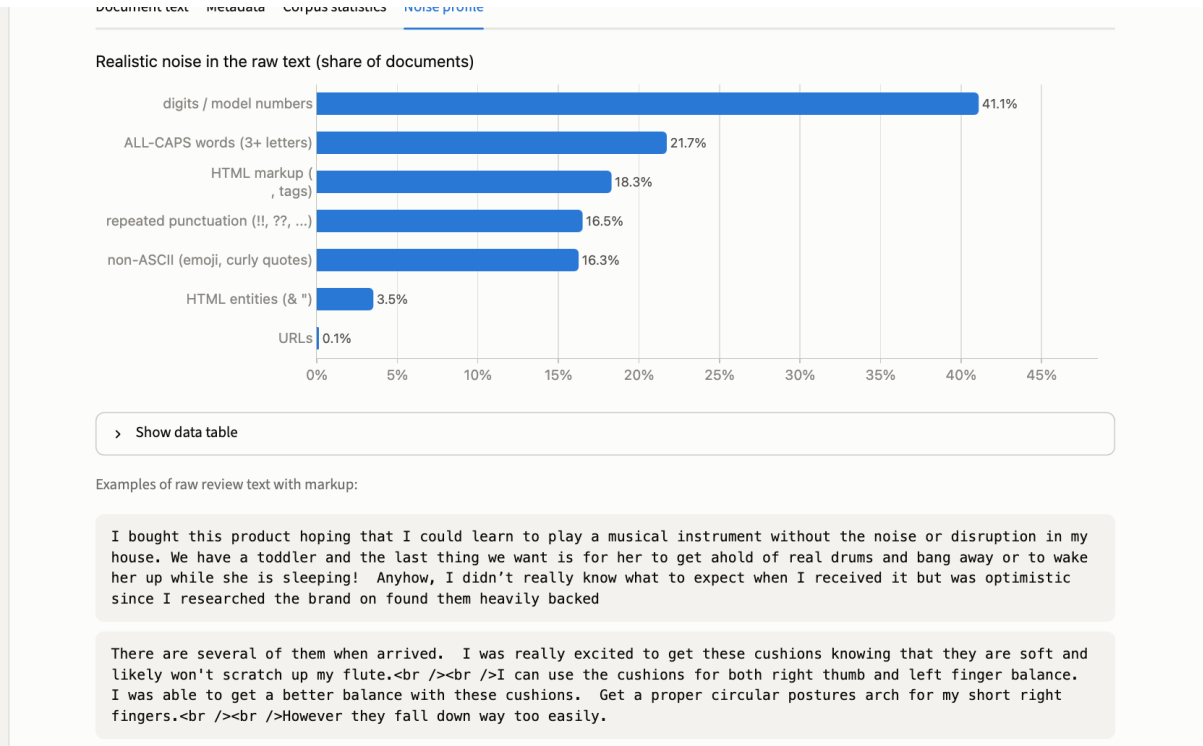


Figure 27. *Noise profile of the raw text on the Data Ingestion page.*

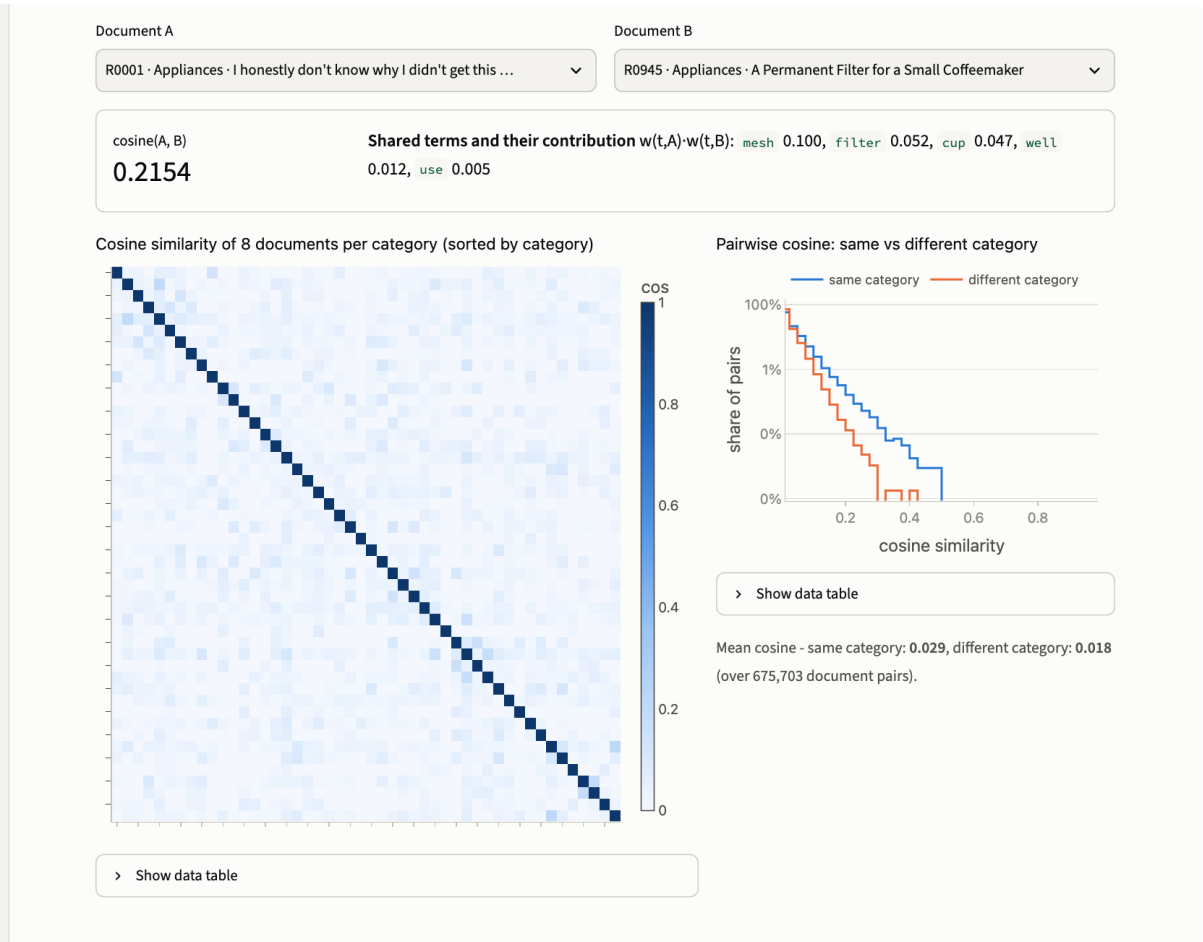


Figure 28. *Cosine similarity tab: pairwise comparison with shared-term contributions, similarity heat map by category and similarity distributions.*

Patterns for domain analysts - which words characterise each department, and what drives negative (1-2★) reviews inside each department.

Vocabulary that distinguishes each department (smoothed log-odds vs all other departments)

category	documents	distinctive terms
Appliances	194	coffe, refriger, stove, burner, drain, keurig, ice, ge
Beauty	194	curl, moistur, lotion, shave, blond, hydrat, shaver, lash
Musical Instruments	194	amp, instrument, guitar, karaok, record, tuner, reverb, violin
Pet Supplies	194	puppi, litter, dog, collar, lb, cage, feed, fur
Software	193	kindl, tax, glitch, hdd, norton, corel, quicken, data
Video Games	194	consol, xbox, wii, gamer, nintendo, razer, scroll, madden

What 1-2★ reviews talk about in each department (log-odds vs 4-5★ reviews of the same department)

category	negative reviews	share negative	what negative reviews talk about
Appliances	45	0.23	sensor, silverwar, return, advertis, compani, disappoint, fact, fork
Beauty	48	0.25	shave, shaver, bristl, cartridg, final, norelco, tape, defect
Musical Instruments	38	0.20	broken, hit, poor, soon, 5mm, anchor, clamp, event
Pet Supplies	42	0.22	crumb, poorli, gourmet, healthi, tell, bare, clip, compani
Software	36	0.19	credit, norton, answer, follow, form, last, turbotax, uninstal
Video Games	40	0.21	ban, crap, found, offic, american, garbag, naga, steam

Themes found by K-Means, ranked by dissatisfaction

cluster	documents	mean rating	share 1-2★	majority category	top terms
C4	277	3.46	0.31	Appliances	work, replac, great, instal, use, new, year, make
C5	244	3.91	0.21	Musical Instruments	qualiti, good, sound, guitar, price, well, like, better
C0	136	3.93	0.18	Beauty	skin, hair, brush, color, use, dri, look, soft
C3	134	4.03	0.17	Beauty	keyboard, hair, use, color, love, mani, even, sturdi
C2	194	4.09	0.17	Pet Supplies	cat, dog, water, love, easi, filter, like, toy
C1	178	3.89	0.17	Software	game, play, fun, app, love, enjoy, time, like

Share of 1-2★ reviews per cluster

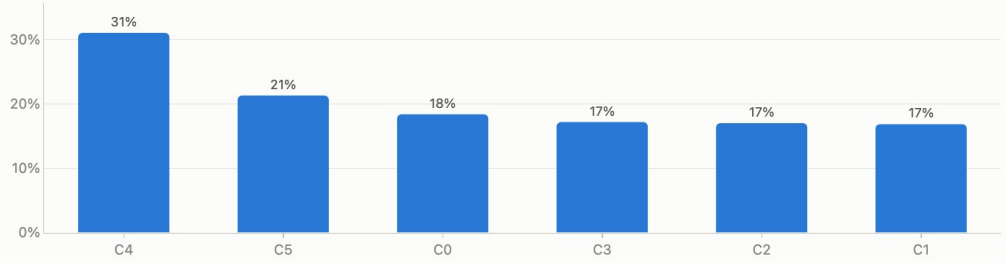


Figure 29. *Corpus insights for analysts: department vocabulary, negative-review themes and K-Means themes ranked by dissatisfaction.*

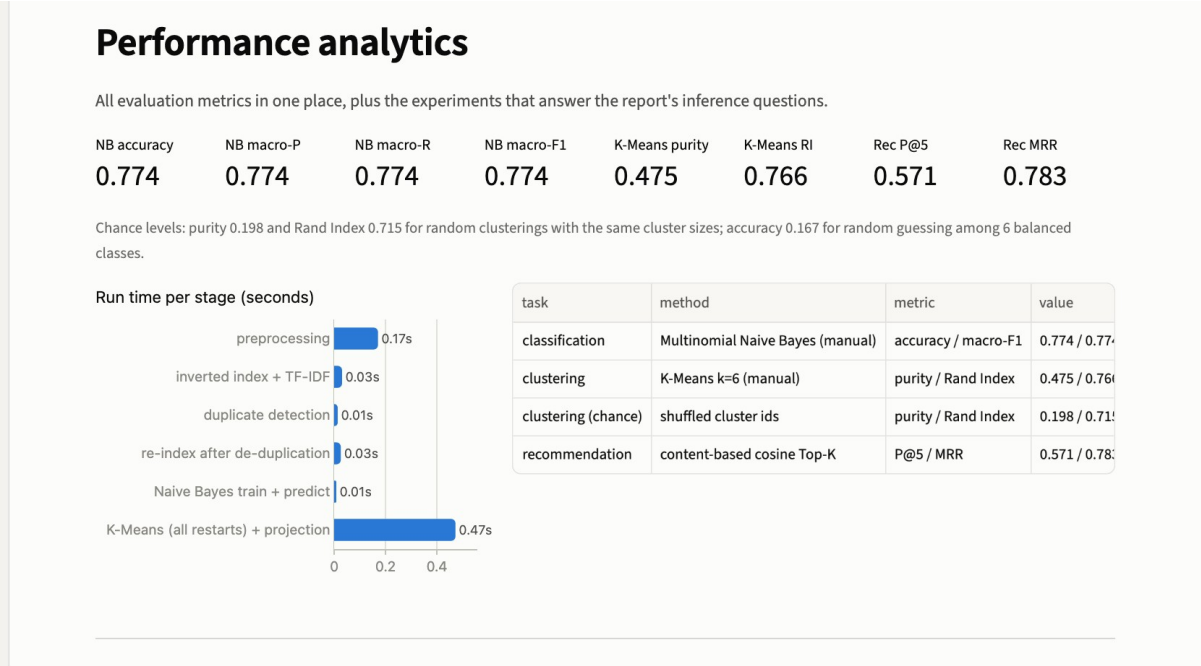


Figure 30. *Performance analytics: scorecard with chance levels, run times per stage and method summary.*

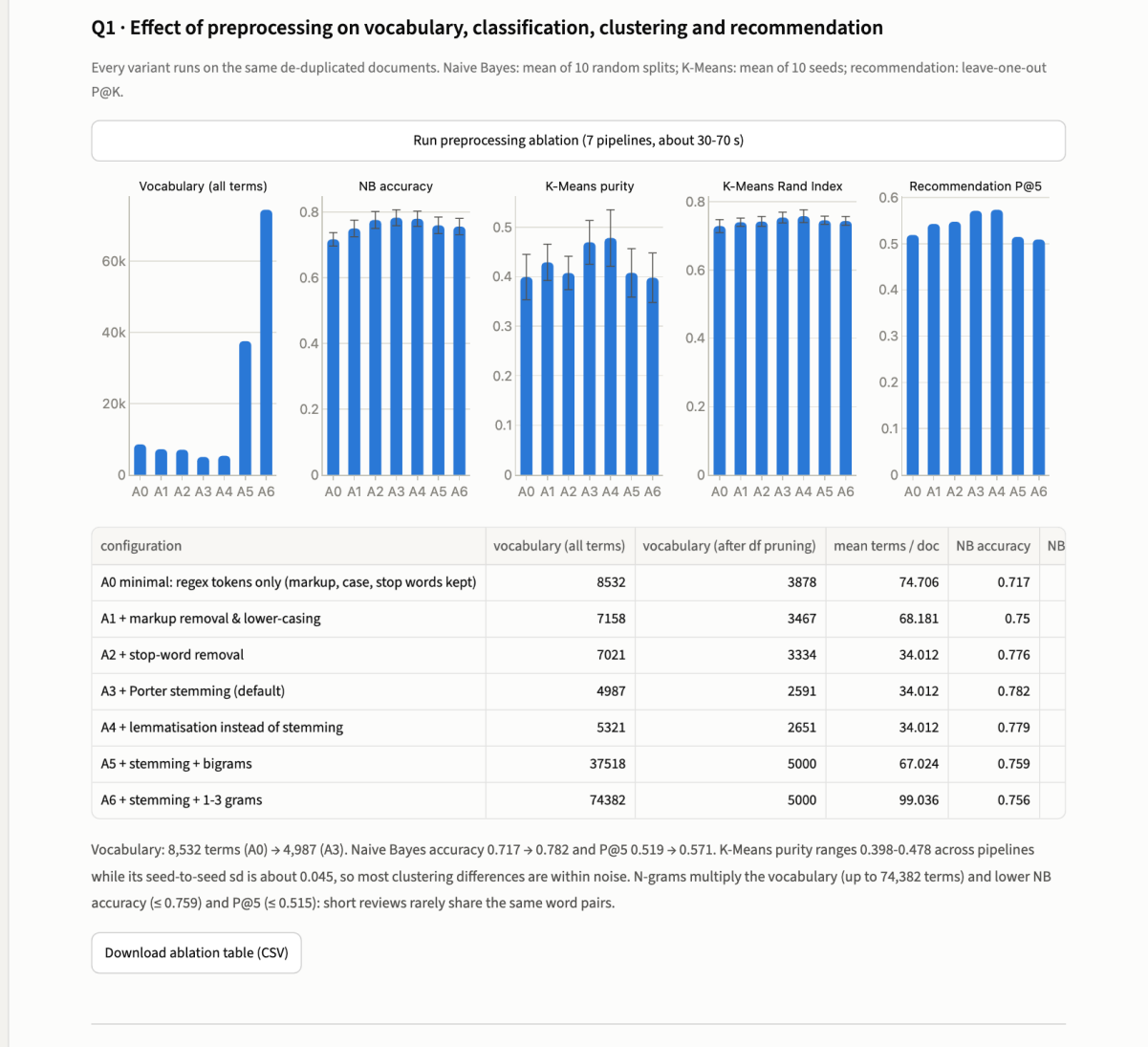


Figure 31. *Performance analytics: preprocessing ablation (Q1).*

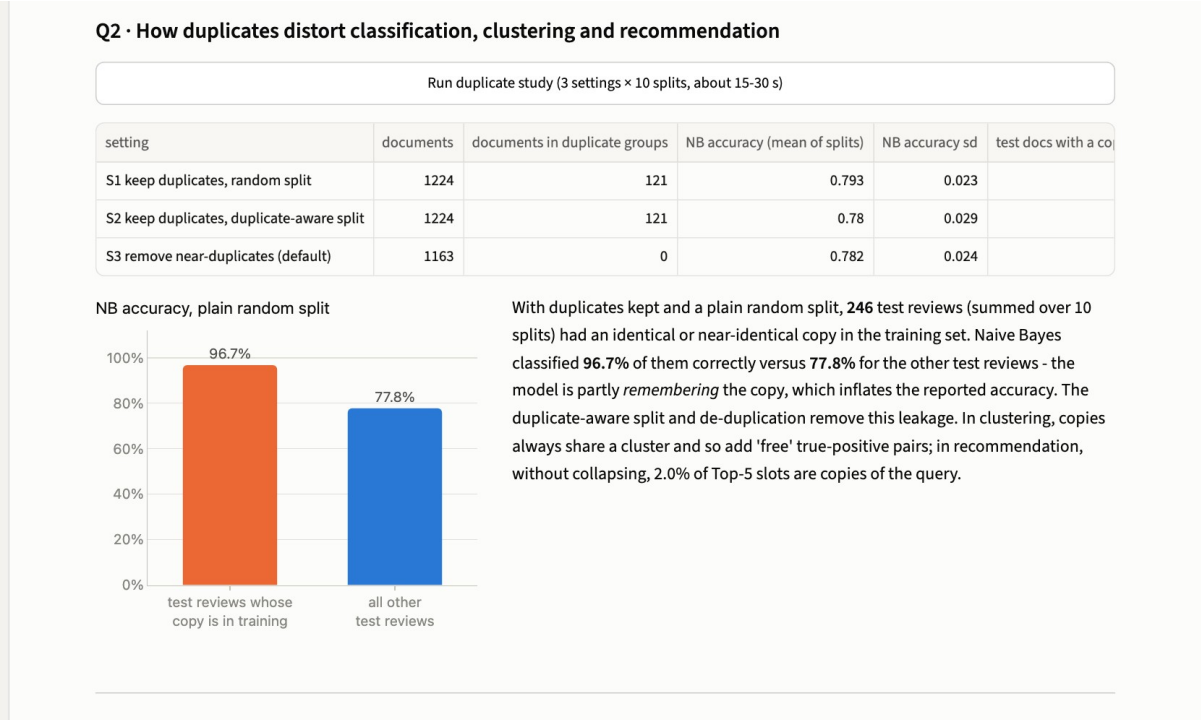


Figure 32. *Performance analytics: duplicate study and leakage chart (Q2).*



Figure 33. *Performance analytics: K-Means variants and k-sensitivity (Q5).*

Appendix B. Test run and environment

Output of python tests/test_core.py (12/12 tests passed):

```
PASS  test_duplicate_detection
PASS  test_index_scoring_equals_matrix_cosine
PASS  test_inverted_index_and_tfidf_by_hand
PASS  test_kmeans_separates_two_blobs
PASS  test_metrics_by_hand
PASS  test_naive_bayes_textbook_example
PASS  test_preprocessing_pipeline
PASS  test_purity_textbook_example
PASS  test_rand_index_textbook_example
PASS  test_rand_index_two_methods_agree
PASS  test_stratified_split_keeps_groups_together
PASS  test_tfidf_textbook_example
```

Environment of the run that produced this report: Python 3.14.6, macOS 27.0.1 arm64, streamlit 1.67.0, pandas 3.1.1, numpy 2.5.4, plotly 7.1.0, nltk 3.10.4. Stemmer: NLTK PorterStemmer; lemmatiser: NLTK WordNet lemmatizer.